Weike Liu

RL,MicroDuck,MuJoCo

TABLE OF CONTENTS

1 Install MuJoCo

2 First Hello World

3 Look at some real stuff

4 What is MuJoCo?

5 Building a robotic arm

5.1 Step 1: Define the robot model (MJCF/XML file)

5.2 Step 2: Parameter configuration file (YAML)

5.3 Step 3: Main Python program (complete development workflow)

5.3.1 Import dependencies and initialization

5.3.2 Load YAML configuration (`load_config` function)

5.3.3 Load MuJoCo model

5.3.4 Enter simulation main loop (with passive viewer)

5.3.5 Main loop internals (`while viewer.is_running()`)

5.3.6 Exit loop and finish

5.3.7 Additional notes

5.4 Step 4: How to run and verify the results

6 Back to the little duck

6.1 MicroDuck main repository

6.1.1 Overall structure

6.1.2 MicroDuck robot control core: robotd

6.1.2.1 robotd's execution flow

6.1.2.2 Part 1: robotd main flow

6.1.2.2.1 Block 1: Parameters and configuration

6.1.2.2.2 Block 2: robotd init is a special path

6.1.2.2.3 Block 3: Preempt robotd's Unix socket

6.1.2.2.4 Block 4: Create RobotState

6.1.2.2.5 Block 5: Create Intents

6.1.2.2.6 Block 6: Create poweroff callback

6.1.2.2.7 Block 7: Start the control thread

6.1.2.2.8 Block 8: Main thread starts IPC server

6.1.2.2.9 How does the IPC server itself work?

6.1.2.2.10 What does the IPC handler do?

6.1.2.2.11 When does the main thread exit?

6.1.2.3 Second main line: 50 Hz Control Loop

6.1.2.3.1 Control Loop initialization phase

6.1.2.3.2 Block 1: Determine current policy

6.1.2.3.3 Block 2: Create Safety

6.1.2.3.4 Block 3: Read startup pose

6.1.2.3.5 Block 4: Create Controller

6.1.2.3.6 Block 5: Establish 50 Hz ticker

6.1.2.3.7 Enter while

6.1.2.3.8 Tick Block 1: Wait for cycle

6.1.2.3.9 Tick Block 2: Read sensors

6.1.2.3.10 Tick Block 3: Coast

6.1.2.3.11 Tick Block 4: Update IMU / odometry / fall status

6.1.2.3.12 Tick Block 5: Read Intent Snapshot

6.1.2.3.13 Tick Block 6: Safety Gate

6.1.2.3.14 Tick Block 7: Handle one-time power request

6.1.2.3.15 Tick Block 8: Handle Skill / Sit / Ground Pick

6.1.2.3.16 Tick Block 9: Handle sound

6.1.2.3.17 Tick Block 10: Handle Mode Switch / Policy Reload

6.1.2.3.18 Tick Block 11: Shutdown / Battery Shutdown

6.1.2.3.19 Tick Block 12: Limp-Fall

6.1.2.3.20 Tick Block 13: Decide whether to enter driving

6.1.2.3.21 Tick Block 14: Handle driving start / stop

6.1.2.3.22 Tick Block 15: Battery voltage compensation

6.1.2.3.23 Tick Block 16: Generate Joint Targets

6.1.2.3.24 Tick Block 17: Controller.step()

6.1.2.3.25 Tick Block 18: Theremin / Chorale / Mouth

6.1.2.3.26 Tick Block 19: Finally enter Safety

6.1.2.3.27 Tick Block 20: Publish RobotState

6.1.2.3.28 Tick Block 21: Record loop health

6.1.2.4 Some key design points of robotd

6.2 robotd's ONNX model declaration, instantiation, and invocation

6.2.1 Define: define an ONNX model object

6.2.2 Create: load the `.onnx` file into a `Session`

6.2.3 `robotd` creates it

6.2.4 Execute: call ONNX

6.2.5 How does `robotd` call it?

6.2.6 Summary of the three actions

6.3 From ONNX file to program output: the complete flow

6.3.1 First understand: what an ONNX file itself is

6.3.2 The complete process (four steps)

7 MicroDuck Training

Chapter 1 Document Description and Training Workflow Overview

1.1 Document Goal and How to Use

1.2 Training Workflow Overview

1.3 Key Concepts and Terminology

1.4 Repository and Related Projects

Chapter 2 Environment Setup

2.1 Hardware Requirements

2.2 Software Requirements

2.3 Clone the Repository

2.4 Install Dependencies

2.5 Verify the Installation

2.6 (Optional) Account Login

Chapter 3 Selecting a Training Task

3.1 View the Task Registry

3.2 Task List

3.3 Task Naming Convention and Backlash Variants

3.4 Robot and Observation/Action Contract

Chapter 4 Smoke Testing

4.1 Why Smoke Test First

4.2 Run the Smoke Test

4.3 Troubleshooting Failed Smoke Tests

Chapter 5 Formal Training

5.1 Start Training

5.2 Training Parameter Description

5.3 Training Monitoring and Result Judgment

5.4 Training Time Budget

5.5 Resume Training from a Checkpoint

5.6 Cross-Task Warm Start

5.7 No-GPU Option: Hugging Face Jobs

Chapter 6 Evaluating Training Results

6.1 Viewer Playback (uv run play)

6.2 Deployment Rehearsal (infer_policy.py)

6.3 Regression Tests

Chapter 7 Export ONNX and Publish

7.1 Export ONNX

7.2 Verify the Exported File

7.3 Publish to Hugging Face Hub

7.4 Real-Robot Installation (robotctl)

7.5 Render a Playback Video (Optional)

Chapter 8 Common Problem Troubleshooting

Chapter 9 Quick Command Reference

Appendix A Key Repository Files and Further Reading

8 Browser simulation tool (visual understanding of architecture operation)

9 Official documentation and feature description (supplementary architecture design philosophy)

10 RL Algorithms

10.1 What problem does reinforcement learning actually solve?

10.2 Why do we need "state, action, reward"?

10.3 What is a policy?

10.4 Why do we need a Value Function?

10.5 Bellman equation: the core mathematical structure of reinforcement learning

10.6 Q Function: why do we still need it?

10.7 The relationship between V and Q

10.8 Optimal value function

10.9 Optimal Q function

10.10 Bellman Optimality Equation

10.11 Why is Q-learning so important?

10.12 The essence of TD Learning

10.13 Exploration and exploitation

10.14 From tabular Q to neural networks

10.15 Reinforcement learning suddenly becomes a function-approximation problem

10.16 The loss function of Deep Q-Network

10.17 Policy Gradient: why can we directly learn the policy?

10.18 The most core formula of Policy Gradient

10.19 Actor-Critic: combining the value function and the policy function

10.20 The intuition of Actor-Critic

10.21 Putting the whole mathematical system of reinforcement learning together

10.22 Understanding reinforcement learning in one sentence

1 INSTALL MUJOCO

On Windows 11, MuJoCo has long been a pure Python package. There is no need to apply for a license or configure complex environment variables.

Open Command Prompt (cmd) or PowerShell.

Run the installation command: pip install mujoco

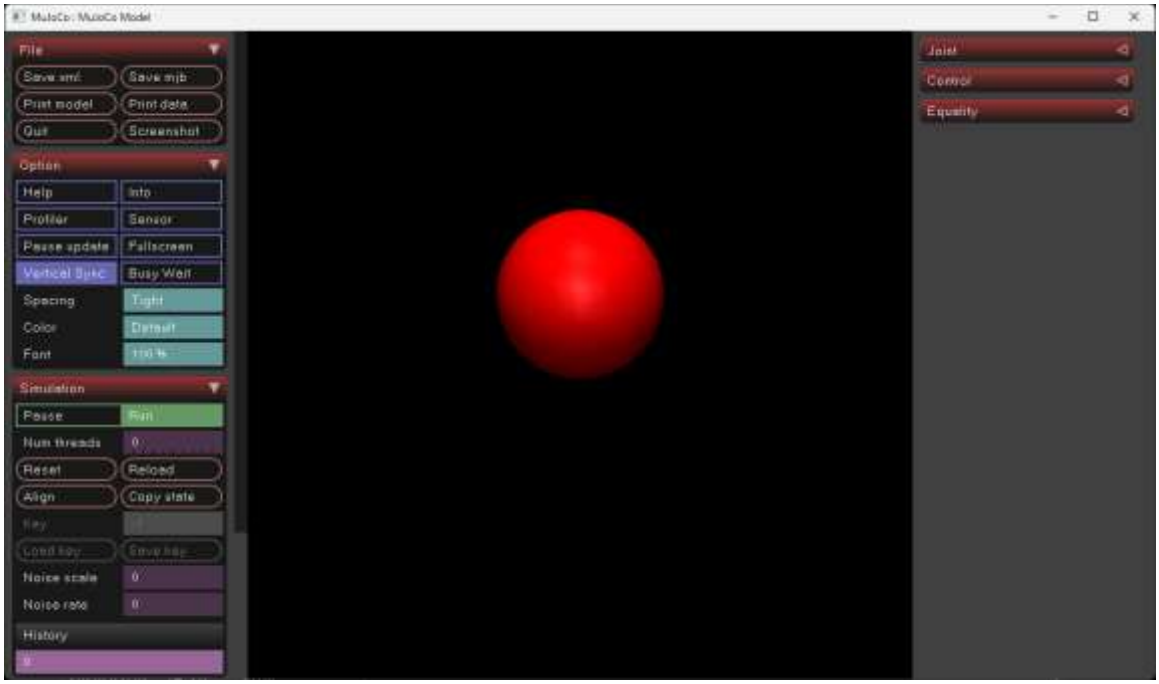
This command will automatically install the core library and rendering dependencies.

After installation, you can verify with: python -c "import mujoco; print(mujoco.__version__)"

2 FIRST HELLO WORLD

HelloWorld.py

Run: python HelloWorld.py, and it immediately appears:



3 LOOK AT SOME REAL STUFF

Install yaml, torch, etc.

Pip install pyyaml

Pip install torch

git clone https://github.com/limxdynamics/humanoid-description.git

Enter the directory, then:

python -m mujoco.viewer --mjcf=HU_D04_description/xml/HU_D04_01.xml

Additionally, there is the Unitree robot: git clone https://github.com/yueruyin/robot-unitree-g1.git

4 WHAT IS MUJOCO?

The core of MuJoCo is a physics engine library written in C. Python is just one of the most popular and convenient ways to call it.

Specifically, you have the following 4 usage approaches:

- Python interface (most commonly used): Installed via pip install mujoco. This is the officially maintained Python binding that wraps the C library into a Python module. This is the method you are currently using to run scripts and view models.
- C/C++ native call: If you are doing low-level robot development or embedded systems, you can directly download MuJoCo's C library (.dll/.so/.dylib) and call its physics calculation functions directly in C/C++ code.
- Command-line tools: After installing the Python package, command-line tools are automatically included. For example, the python -m mujoco.viewer --mjcf=xxx.xml you used before essentially calls the built-in visualization program.
- Bindings for other languages: The community also provides Julia (MuJoCo.jl), Rust (mujoco-rs), and even MATLAB interfaces, covering most mainstream programming languages.

Summary: Python is the preferred choice for getting started and doing reinforcement learning (RL), but MuJoCo itself is very "general-purpose" and not tied to any language.

1. Core Positioning: Library vs. Platform

MuJoCo is a "library": It's like OpenCV (computer vision library) or FFmpeg (audio/video library). It is a set of low-level computation tools with no "main interface". Your code is the "commander", calling its API to compute physics, contact forces, and joint angles, then you draw the results with other tools. It works entirely around code.

Isaac Sim is a "platform/application": It's like Unity or Unreal Engine. It is a massive graphical integrated environment. Most of your time is spent dragging and dropping models, building scenes, setting lights in the GUI, and clicking buttons to run simulations. Code (Python) is just a "plugin" to assist GUI operations or write complex logic.

2. The "trap" of the GUI (something many people misunderstand)

You might ask: "But doesn't python -m mujoco.viewer also pop up a window (GUI)?"

Note the distinction: The window that comes with MuJoCo is essentially a "debug monitor", not a "development platform".

Isaac Sim's GUI: You work inside it (building scenes, modifying physics parameters, viewing render effects).

MuJoCo's viewer: You just use it as a display (to see how the robot is actually running, whether its legs are shaking, whether it has fallen). Once you close this viewer, your Python script can still compute physics data in the background (such as logging or training AI), while Isaac Sim ends the entire simulation environment once you close the GUI.

5 BUILDING A ROBOTIC ARM

5.1 STEP 1: DEFINE THE ROBOT MODEL (MJCF/XML FILE)

MuJoCo uses the MJCF (MuJoCo XML) format to describe robots. This file defines the geometry, joints, mass, colors, etc. of the robotic arm. Create a two_link_arm.xml file.

5.2 STEP 2: PARAMETER CONFIGURATION FILE (YAML)

YAML is used here to store "control algorithm parameters", avoiding hardcoding them in Python code: config.yaml.

5.3 STEP 3: MAIN PYTHON PROGRAM (COMPLETE DEVELOPMENT WORKFLOW)

run_arm.py, this is the core code. It integrates model loading, YAML reading, control loop, and visualization all together.

The program has excellent comments and is very easy to read.

This Python program (`run_arm.py`) implements a two-link robotic arm trajectory tracking simulation based on the MuJoCo physics engine. It sets joint target trajectories through a YAML configuration file and uses the built-in `position` actuator in the XML model (with built-in PD control) to directly send desired angles, thereby driving the robotic arm's motion. Below is a step-by-step explanation of its workflow in execution order:

5.3.1 Import dependencies and initialization

Import `time`, `numpy`, `yaml`, `mujoco`, and its `viewer` submodule. These libraries are used for time control, numerical computation, configuration file parsing, physics simulation, and visualization, respectively.

5.3.2 Load YAML configuration (`load_config` function)

Attempt to open the `config.yaml` file and parse it into a dictionary.

- If the file does not exist, use default parameters (containing only trajectory amplitude, frequency, and phase difference) and display a warning.

- The configuration content is used to generate the target angle trajectories for the two joints.

5.3.3 Load MuJoCo model

Read the XML model file via `mujoco.MjModel.from_xml_path("two_link_arm.xml")`.

If the file fails to load, the program prints an error and exits.

Then create an `MjData` object to store simulation state (position, velocity, control variables, etc.).

5.3.4 Enter simulation main loop (with passive viewer)

Use `mujoco.viewer.launch_passive()` to start a passive (non-blocking) visualization window, passing in the model and data.

Record the start time `start_time` for calculating the elapsed simulation time `elapsed`.

5.3.5 Main loop internals (`while viewer.is_running()`)

Each iteration executes the following steps:

a. Calculate desired joint angles

- Based on parameters such as `amplitude1` and `freq1` in the configuration, use sine functions to generate the target angles for the two joints:

  - `target1 = A1 * sin(2π * f1 * t)`

  - `target2 = A2 * sin(2π * f2 * t + ϕ2)`

- Here `t` is the simulation runtime (physical time, not CPU time).

b. Directly send target angles

- Assign the calculated `target1` and `target2` to `data.ctrl[0]` and `data.ctrl[1]`.

- Key point: The joint actuator type in the XML model is `<position>`, which has `kp` (proportional gain) and `kv` (velocity gain) defined internally. Therefore, MuJoCo automatically converts angle errors into torques, **without needing to manually compute the PD control law in Python**.

c. Step the physics engine

- Call `mujoco.mj_step(model, data)` to advance the simulation by one step, updating all states including position, velocity, and torque.

d. Sync visualization

- Call `viewer.sync()` to render the current data to the display window.

e. Control simulation speed

- Use `time.sleep(model.opt.timestep)` to make the program sleep for one physics timestep (typically a few milliseconds), ensuring the simulation runs at roughly real-time speed (real-time simulation) and preventing the loop from running too fast.

5.3.6 Exit loop and finish

- When the user closes the viewer window or presses the stop key, `viewer.is_running()` becomes `False`, and the loop terminates.

- Print `"Simulation ended"` and the program exits.

5.3.7 Additional notes

- The entire process is a closed loop of **"Configuration → Model loading → Real-time trajectory generation → Direct position control → Physics simulation → Rendering"**.

- This design places low-level control (PD parameters) in the XML, while the upper-level Python only handles trajectory planning, achieving separation of control logic from physics simulation for easier debugging and parameter adjustment.

- The real-time performance of the simulation depends on `time.sleep`, but the actual physics timestep is determined by `model.opt.timestep` (usually fixed), and `sleep` is only for throttling. If the computer performance is insufficient, the simulation may slow down, but it will not affect physics accuracy.

5.4 STEP 4: HOW TO RUN AND VERIFY THE RESULTS

Run the script: python run_arm.py

You will see: a green + blue robotic arm, with the base rotating (joint 1), while the upper arm swings up and down (joint 2), and the orange ball at the end traces an elegant "8" figure trajectory. You can drag to rotate the view with the mouse, right-click to pan, and scroll to zoom.

Through this example, understand MuJoCo development's "four-step method"

Stage	File/Code	Role
① Model definition	*.xml (MJCF)	This is the robot's "physical blueprint", defining geometry, mass, joint types (hinge/ball/free), friction, colors, etc. The essence of

		MuJoCo lies in tuning the XML parameters.
② Parameter configuration	*.yaml	This is the "tuning sheet for control algorithm parameters". It moves frequency, amplitude, and gains out so you can tweak them repeatedly during experiments without frequently changing the Python code.
③ Core logic	Python script	Responsible for loading (model + YAML), computing (target trajectories), executing (assigning data.ctrl), and stepping (mj_step). This is where you write your algorithms.
④ Visualization loop	viewer.launch_passive	The passive loop keeps you in control — compute control first, then step the physics, and finally refresh the view. This is more suitable for algorithm development than launch (fully automatic).

Advanced tips

Adjust parameters: Modify amplitude or freq in config.yaml, save and re-run to observe changes in the robotic arm's swing, without changing Python.

Export data: Add a line print(data.body('tip').xpos) in the loop to print the end-effector's 3D position in real-time for trajectory analysis.

Change to joint space following: If you want it to maintain a fixed posture, just hardcode target1 = 0.5.

This framework covers the basic structure of virtually all MuJoCo simulation projects. You can replace two_link_arm.xml with the Unitree G1 model and replace the sine wave with a reinforcement learning policy to upgrade to an advanced project.

6 BACK TO THE LITTLE DUCK

If you are not familiar with ONNX yet, you can first read 6.3.

MicroDuck has these official open-source repositories:

- Official main repository, the core repository of Microduck, containing the SDK and robot body software.

Address: `https://github.com/pollen-robotics/microduck/tree/main`

License: Apache-2.0, which means you are free to use, modify, copy, and distribute the code. Including:

Commercial use: You can use the code in commercial software or internal projects.

Closed-source distribution: You can integrate the code into your own products without open-sourcing your own source code.

Relicensing: You can distribute the entire work (including Apache-2.0 code) under other licenses.

Obligations you must comply with:

Retain notices: When distributing code, you must retain all copyright notices, license declarations, and attribution information in the NOTICE file from the original code.

Mark modifications: If you modify files, you need to clearly mark where the changes were made.

Include license: When redistributing, you need to include a copy of the Apache-2.0 license.

- Official reinforcement learning repository, this repository focuses on reinforcement learning and sim-to-real deployment tools. Address: `https://github.com/pollen-robotics/microduck_rl`
- Official related tools

Microduck Studio: a local web development studio for debugging, visualizing, and orchestrating Microduck workflows. Address: `https://github.com/microai-lab/microduck-studio`

- Additionally, there is a third-party replica project, a community-initiated third-party replication project aimed at reconstructing assembly drawings, CAD assemblies, etc. from the officially published simulation models and source code. Address: `https://github.com/fanhao375/microduck-replica`

Note that Microduck's software is open source (Apache-2.0), but the hardware itself is not open source.

6.1 MICRODUCK MAIN REPOSITORY

Address: https://github.com/pollen-robotics/microduck/tree/main. This repository contains the software running on MicroDuck's RK3566 processor. MicroDuck sometimes refers to it as "runtime" software.

Microduck is not a "single program", but rather a Rust-based multi-process robot operating system/runtime running on the robot's RK3566 Linux board.

The core is robotd, which handles the actual robot closed-loop control; other daemons [1] around it handle configuration, Bluetooth, gamepad, camera/remote communication, ToF, and software updates. All control services communicate via Unix Socket + JSON-RPC 2.0/NDJSON.

6.1.1 Overall structure

The repository itself is a Rust workspace. Currently, the Cargo workspace [2] has 19 members, including daemon, control algorithm library, CLI, IPC protocol, tools, etc.

1In Linux, a daemon is a special process that runs in the background, silently providing services without any interactive interface. You can think of it as a "backstage worker" or "system attendant" in your computer: you don't need to talk to it directly (no GUI or command-line interaction); it is always on standby (continuously running in memory); when there is a request, it works (responding to client calls); when there is no request, it idles (using minimal CPU resources). One point that is easy to confuse:

daemon: runs in the background, no interaction, provides system services.

While "service" is more of a "functional-level" term. The daemon is the concrete process that implements a service; the two are often used interchangeably. For example, the "SSH service" is provided by the sshd daemon.

How to create your own daemon (briefly): if you write it in C, the standard steps are:

fork() creates a child process and the parent exits; the child calls setsid() to create a new session; close the file descriptors (stdin/stdout/stderr); change the working directory to /; set the file permission mask umask(0), etc.

However, nowadays it is recommended to write a systemd service unit file directly and let systemd manage it for you, avoiding these low-level details.

2Cargo Workspace is a proper noun specific to the Cargo tool in the Rust ecosystem. It is a core feature designed by the official Rust team for managing multiple interrelated packages, with no fully equivalent native concept in other programming languages' build systems. Core definition:

A Cargo Workspace is a collection of interrelated Rust packages (crates) developed collaboratively. All member packages share the same Cargo.lock dependency lock file and the same target compilation output directory, fundamentally avoiding dependency version fragmentation and repeated compilation issues in multi-package development.

Core features:

Unified multi-package management: you can split a large project into multiple independent crates with single responsibilities, all maintained collaboratively in the same workspace. A typical case is the official Rust rustc compiler repository — the entire project is a giant Cargo Workspace managing nearly 50 interdependent sub-crates.

Shared configuration and dependencies: [workspace.dependencies] in the root Cargo.toml can define globally shared dependencies, and all sub-crates can directly inherit these dependency versions without repeatedly writing configuration, greatly reducing the probability of dependency version conflicts.

It has these "entry points":

- Linux/systemd service entry point

The programs that truly run persistently after robot startup are mainly:

Entry	Type	Core role
robotd	daemon	Robot motion control core
configd	daemon	WiFi, robot identity, pairing, system configuration
updaterd	daemon	Software/model updates, verification, rollback
btd	daemon	BLE API entry
padd	daemon	Gamepad entry
mediad	daemon	Camera/microphone/WebRTC/remote API
tofd	daemon	ToF depth sensor

These 7 are the most core service divisions of the entire runtime. The architecture documentation also explicitly states that systemd is their supervisor, responsible for lifecycle, crash restart, startup order, and watchdog.

- User/developer CLI entry point

It also has two very important CLIs (Command Line Interfaces): robotctl, duckctl.

✧ Robotctl: This is the management CLI executed directly on the robot's Linux board.

It can perform operations such as:

robotctl version

robotctl health

robotctl monitor

robotctl configure

robotctl policy ...

Flexible build control: you can run cargo build at the root to compile all member packages at once, or use the -p package-name parameter to build, test, or publish a specific sub-crate individually, balancing overall build efficiency and single-package iteration flexibility.

Two manifest modes: the virtual manifest mode (the root Cargo.toml contains only [workspace] configuration and is not itself an independent crate) and the mode where the root is a member package, adapting to different project organization needs.

robotctl robot ...

robotctl pad ...

robotctl update ...

robotctl wifi ...

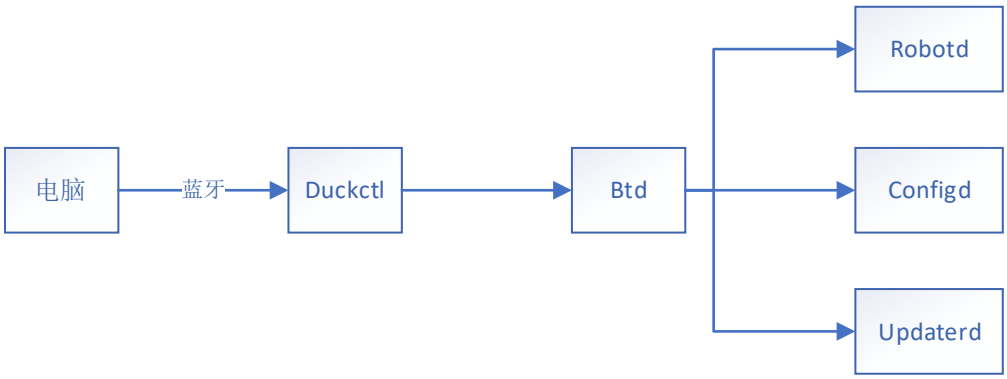
robotctl system ...

robotctl voice ...

The official cheat sheet defines robotctl as the main control/diagnostic entry point on the robot.

- ✧ Duckctl: this is the CLI on the computer outside the robot. It finds Microduck via Bluetooth, then sends the same JSON-RPC requests as the robot's internal ones.

Therefore:



The code comments explicitly state that duckctl is a "stand-in for the phone app", used only for development and not deployed to the robot.

6.1.2 MicroDuck robot control core: robotd

It is the real-time control daemon that truly makes the duck move. robotd executes a 50Hz control loop, drives 15 servos, and runs neural network policies. robotd is the only daemon that can control motors, and the reason will be explained later. MicroDuck is written in Rust, so many concepts involve Rust-specific concepts, such as slot, tokio, etc., as described below.

6.1.2.1 robotd's execution flow

MicroDuck's robotd is written in Rust, and its entry point is:

#[tokio::main]

async fn main() -> ExitCode

After program startup, main()

1. reads robotd.toml
2. creates /run/robotd.sock
3. creates RobotState + Intents
4. starts control thread
5. the main tokio runtime runs the IPC server

A very noteworthy point: the Control loop started in step 4 is a thread, not sharing a runtime with the step 5 IPC. The code specifically creates:

OS thread: control

```
    |
    └──── current-thread tokio runtime
              |
              └──── 50 Hz loop
```

The reason is that Dynamixel serial I/O is blocking. If control loop and IPC are placed in a normal tokio runtime:

IPC request

```
    ↓
```

tokio worker

```
    ↓
```

blocking serial read

```
    ↓
```

The entire worker gets occupied, and other tasks in the IPC loop may affect the control cycle.

Therefore, consider main.rs as a "main flow + control loop", focusing only on two main lines:

1. robotd main flow: How the process establishes services, creates shared state, starts control threads, and provides IPC after startup.
2. 50 Hz control loop: The execution order of each tick in the control thread.

Below is based on the current main branch code. robotd/src/main.rs currently has about 7600 lines, but the core areas that truly determine the runtime main flow are concentrated in main(), spawn_control_thread(), and control_loop().

6.1.2.2 Part 1: robotd main flow

main() is the true process entry point. The code starts from:

#[tokio::main]

async fn main() -> ExitCode

What is actually done here is a series of initialization and assembly, not controlling the robot. In code order:

main()

├── 1. Parse command line arguments

├── 2. Initialize logging

├── 3. Load robotd.toml

├── 4. Handle --port / --no-policy

├── 5. If it's `robotd init`, follow the separate init flow

├── 6. claim Unix socket

├── 7. Create RobotState

├── 8. Create Intents

├── 9. Create poweroff callback

├── 10. Create control thread

├── 11. Start IPC server

└── 12. Wait for server / SIGTERM

So main() essentially assembles the runtime environment for robotd, then starts two main execution entities: control thread + IPC server.

6.1.2.2.1 Block 1: Parameters and configuration

Params is static configuration. For example: bus port, control period, policy, safety, deadman, gain, limp fall, audio, mode, etc. The things that change continuously during runtime are not placed in Params, but rather in: RobotState, Intents, Controller - these three different layers.

6.1.2.2.2 Block 2: robotd init is a special path

robotd init is a CLI command. Its execution follows a separate path. After initialization completes, the robot is in "establishing the robot's known posture" state, rather than the normal continuous control service.

6.1.2.2.3 Block 3: Preempt robotd's Unix socket

Only then is it allowed to start the subsequent control thread. The code explicitly requires owning the endpoint before starting threads that may access motors.

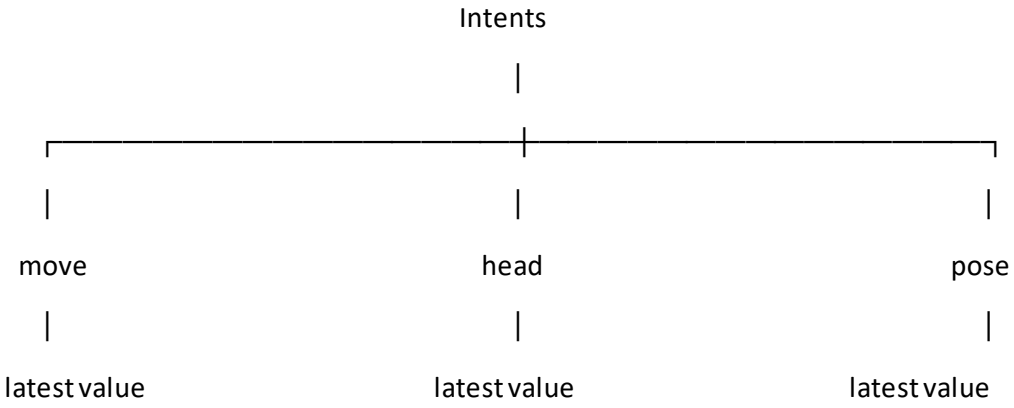
6.1.2.2.4 Block 4: Create RobotState

Next: let state = Arc::new(RobotState::new(...)); This RobotState is very important. It is not a "Mutex of all the robot's current states". It is primarily: the state publishing channel from control loop → IPC.

6.1.2.2.5 Block 5: Create Intents

Next: let intents = Arc::new(Intents::new());

This object is the command exchange area between IPC and the control loop. Note that it is not an ordinary command queue. It is more like:



For example, the client continuously sends:

robot.move(vx=0.1)

robot.move(vx=0.2)

robot.move(vx=0.3)

The control loop does not consume them one by one:

0.1

0.2

0.3

Instead, at each tick:

let snapshot = intents.snapshot();

It gets the command snapshot at this moment.

Therefore:

IPC produces command

↓

Intents

↓

20ms snapshot

↓

Used in current control cycle

This is a very critical architectural point.

6.1.2.2.6 Block 6: Create poweroff callback

Then: let poweroff: PowerOff = Arc::new(...). It does not participate in normal control. It only gives the control loop the ability to: "if shutdown is decided, truly execute systemctl poweroff".

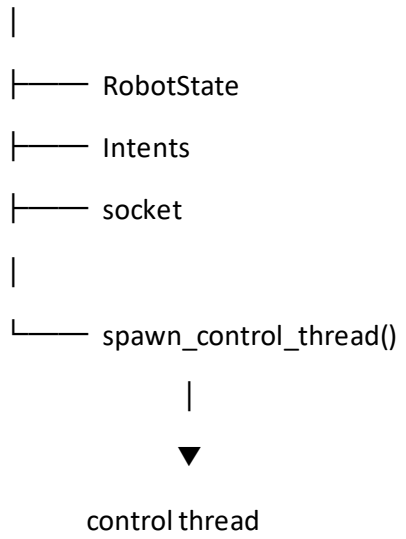
So the control loop does not directly depend on shell/systemd logic, but instead receives a: PowerOff callback.

This also facilitates testing.

6.1.2.2.7 Block 7: Start the control thread

Then: let control = spawn_control_thread(...) — this is the dividing point of the entire main(). Up to here:

main thread

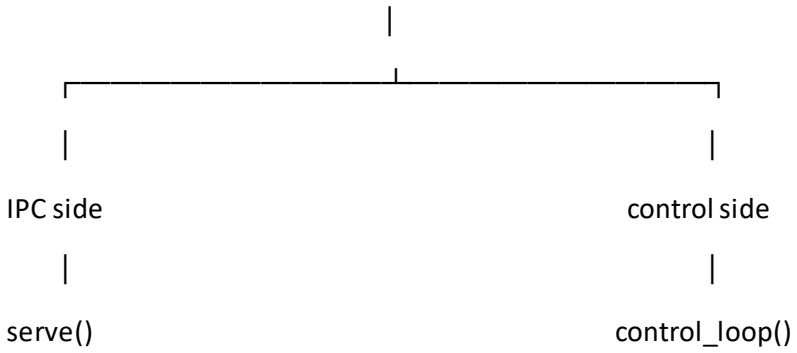


control thread

Note: The real robot control starts here and enters another execution context.

6.1.2.2.8 Block 8: Main thread starts IPC server

After the control thread starts, the main thread continues: let serving = serve(...). That is to say:



Then:

```
tokio::select! {
    result = serving => ...
    _ = shutdown() => ...
}
```

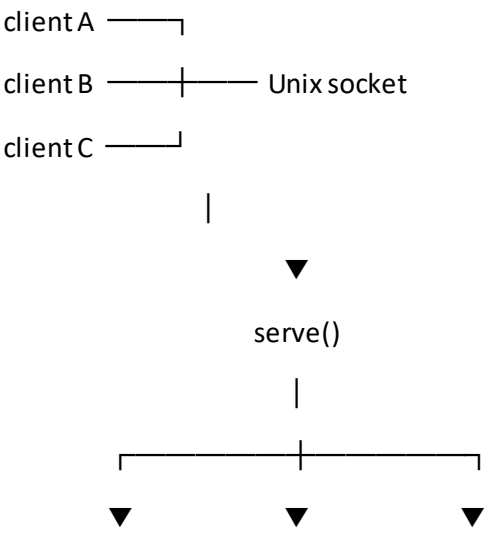
The main thread waits here.

6.1.2.2.9 How does the IPC server itself work?

serve() is very simple:

```
loop
 ├─── listener.accept()
 ├─── Get UnixStream
 └─── tokio::spawn(handle(...))
```

That is to say:



handle handle handle

Each client can be handled independently. The code is accept → clone state/intents → tokio::spawn(handle), meaning each client is handled by its own handle.

6.1.2.2.10 What does the IPC handler do?

handle() receives JSON-RPC/NDJSON requests. It has two types of tasks:

Type 1: Queries

For example:

robot.health

robot.state

robot.policies

robot.safeToRestart

These mainly read: RobotState

Type 2: Control

For example:

robot.move

robot.head

robot.init

robot.relax

robot.do

robot.setMode

...

These do not directly control motors. Instead, requests generate intents to the control thread. Therefore, they ultimately have to wait for the next tick of the control loop.

6.1.2.2.11 When does the main thread exit?

When: SIGTERM or: IPC server error, the main flow ends. Then:

state.shutdown.store(true, ...)

Notify the control loop: "Stop.", then:

control.join()

Wait for the control thread to truly exit.

Finally:

remove_file(socket) removes the Unix socket.

There is also a very important detail here: it does not directly kill the control loop. The code requires:

shutdown = true

↓

control loop ends current tick by itself

↓

Exit

↓

join

The reason is that it cannot forcefully abort in the middle of a Dynamixel bus transaction.

6.1.2.3 Second main line: 50 Hz Control Loop

This is the core of robotd. Entry point: async fn control_loop<T: RobotIo>(...), code starts around line 1600. After it starts, it does not immediately:

while true

policy

Instead, it first establishes the control environment.

6.1.2.3.1 Control Loop initialization phase

In code order:

control_loop()

├── 1. Prepare policy configuration

├── 2. Clean up unloadable overrides

├── 3. resolve policy

├── 4. Create Safety

├── 5. Read and adopt startup pose

├── 6. Create Controller

├── 7. Create 50Hz ticker

└── 8. Initialize control loop state

6.1.2.3.2 Block 1: Determine current policy

6.1.2.3.3 Block 2: Create Safety

6.1.2.3.4 Block 3: Read startup pose

6.1.2.3.5 Block 4: Create Controller

6.1.2.3.6 Block 5: Establish 50 Hz ticker

Then: let mut ticker = tokio::time::interval(period); Normally: period = 20ms, then:

```
ticker.set_missed_tick_behavior(
    MissedTickBehavior::Skip
);
```

This is very important. It means:

tick 1

tick 2

tick 3 ← This one was too slow, missed

tick 4

It will not catch up:

tick 3

tick 3

Instead, it directly enters the latest time point. The control system does not want to accumulate old commands. The code also explicitly states that Skip is chosen to avoid backlog and time drift.

6.1.2.3.7 Enter while

Code:

```
while !state.shutdown.load(...) {
    ticker.tick().await;
```

This is the robot's true: 50 Hz main loop. Below, you must understand strictly in code order.

6.1.2.3.8 Tick Block 1: Wait for cycle

Each time: ticker.tick().await; waits for the next control cycle. Then: let tick_start = Instant::now(); records this tick's start point.

6.1.2.3.9 Tick Block 2: Read sensors

6.1.2.3.10 Tick Block 3: Coast

If reading sensors fails. Code: let sensors = coast.sample(fresh); Meaning:

fresh read succeeded

↓

Use fresh

fresh read failed

↓

Temporarily use last good sample

Maximum COAST_TICKS = 3, which is approximately: 60 ms

6.1.2.3.11 Tick Block 4: Update IMU / odometry / fall status

6.1.2.3.12 Tick Block 5: Read Intent Snapshot

6.1.2.3.13 Tick Block 6: Safety Gate

This resolves: "Does the user want the robot to move in a way that has already timed out, or should it be restricted?"

6.1.2.3.14 Tick Block 7: Handle one-time power request

6.1.2.3.15 Tick Block 8: Handle Skill / Sit / Ground Pick

Requests such as ground_pick, sit_toggle, skill do not directly produce joint targets. They only change: the Controller's internal state. For example:

robot.do kick

↓

controller.start_skill()

↓

active = kick

Then the real target is only generated later by: controller.step(). This is very important: IPC request is "changing controller state", not "directly outputting motor targets".

6.1.2.3.16 Tick Block 9: Handle sound

6.1.2.3.17 Tick Block 10: Handle Mode Switch / Policy Reload

6.1.2.3.18 Tick Block 11: Shutdown / Battery Shutdown

6.1.2.3.19 Tick Block 12: Limp-Fall

6.1.2.3.20 Tick Block 13: Decide whether to enter driving

This is the most important condition judgment in the entire loop:

```
let driving =
    snapshot.enabled
    && bringup == Bringup::Ready
    && controller.is_some()
    && !in_limp_fall
    && sensors.is_some()
    && imu_warm
    && !powered_off;
```

Only when all conditions are met does the policy truly have control of the robot.

6.1.2.3.21 Tick Block 14: Handle driving start / stop

6.1.2.3.22 Tick Block 15: Battery voltage compensation

The purpose is to maintain the effective control response of the servos as much as possible.

6.1.2.3.23 Tick Block 16: Generate Joint Targets

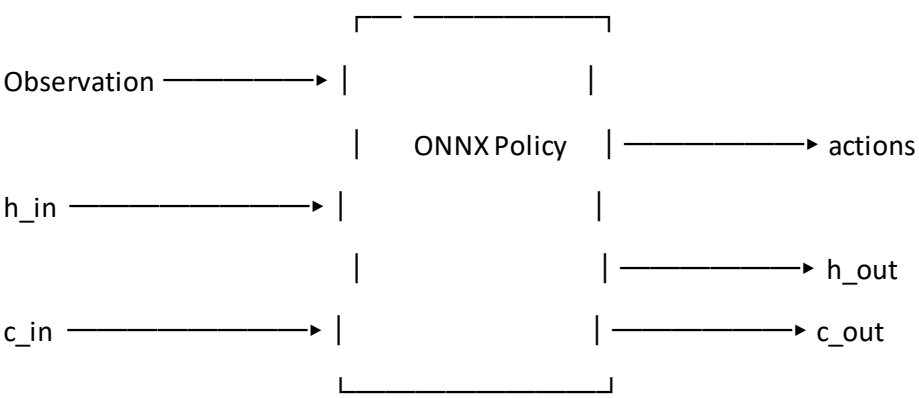
6.1.2.3.24 Tick Block 17: Controller.step()

In robotd's control loop, when the robot is truly driving, it enters:

```
controller.step(
    sensors,
    &command,
    body_active,
    period.as_secs_f64(),
    scale_mult,
)
```

That is to say: the control loop itself does not directly call ONNX. It passes sensors, commands, etc. to the Controller. The Controller is the control algorithm layer. The file robotd/src/control.rs even explicitly states at the beginning: Controller is located between RobotIo::read and safety apply, only does computation, has no IO handle itself, and therefore cannot directly control motors.

So the model is actually:



The Policy does not directly learn "absolute joint angles", but rather learns action offsets relative to the default/home pose.

6.1.2.3.24.1 Controller Block 1: Select which Network

6.1.2.3.24.2 Controller Block 2: Construct Observation

6.1.2.3.24.3 Controller Block 3: Action → Joint Target

6.1.2.3.24.4 Controller Block 4: Low-pass

6.1.2.3.24.5 Controller Block 5: Update its own state

6.1.2.3.25 Tick Block 18: Theremin / Chorale / Mouth

6.1.2.3.26 Tick Block 19: Finally enter Safety

Policy

↓

targets

↓

Expression layer modification

↓

Safety.apply()

↓

RobotIo

"Writing motors" is in Controller.step()

↓

generate joint targets ← motors are not written yet here

↓

Safety.apply()

↓

RobotIo.write() ← This is where motors are truly written to

↓

Dynamixel

RobotIo executes the actual Dynamixel write operation. Since these are serial motors, it actually calls Dynamixel's serial output.

6.1.2.3.27 Tick Block 20: Publish RobotState

6.1.2.3.28 Tick Block 21: Record loop health

6.1.2.4 Some key design points of robotd

1. After startup, it does not automatically make the robot stand up. After startup, Duck is in a limp state, preparing for update.
2. External control does not directly control motors. For example, the keyboard does not send commands directly to motors, but rather:

   ↓

   robot.move

   ↓

   ↓

   ↓

   That is, the keyboard sends robot movement commands via IPC (inter-process communication) as intents to the control loop (the control thread mentioned earlier), which then controls the motors.gamepadIPCIntentscontrol loop

3. Intents is the "buffer layer" between IPC and control loop
   intents.rs is a very important file. It does not use mutexes [3], but rather multiple independent slots [4].
   Every 50 Hz tick, the control loop executes:
   let snapshot = intents.snapshot();
   It gets this tick's input all at once. After getting the Intent:
   let (gated, deadman) =
   safety.gate(snapshot.command, snapshot.twist_age);
   Intent
   
   Safety.gate()
   
   Is twist timed out?
   Is movement allowed?
   What restrictions are there?
   If gamepad stops sending: robot.move
   Finally:
   twist_age > deadman timeout
   it will be cut off by the safety gate. So gamepad disconnection will not cause:
   Last vx=1.0

   Robot permanently maintains vx=1.0
   It will enter deadman instead.

3Mutex (mutual exclusion lock) is a core mechanism for thread synchronization in multithreaded programming, whose full name is Mutual Exclusion. Its core role is to ensure that only one thread can access a shared resource at a time, completely avoiding data race problems caused by concurrent multithreaded operations.

Core principle and role

Mutual exclusion access: it is like a lock with a unique key. Before entering the critical section that accesses shared resources, a thread must acquire the lock, and release it after the operation completes; other threads attempting to acquire the already-held lock will be blocked until the lock is released.

Resolving data races: when multiple threads read and write the same shared memory (such as global counters or shared files) simultaneously, the lack of a synchronization mechanism makes results unpredictable. Mutex forces serialized access, ensuring the logical atomicity of shared data operations and avoiding data corruption.

Strict ownership: mutexes are thread-affine; only the thread holding the lock can release it. Releasing a lock from another thread directly triggers an exception or program error.

Different programming languages and systems provide mutex types adapted to different scenarios:

Type	Core features	Applicable scenarios
Plain mutex (std::mutex)	Basic and non-recursive; locking twice deadlocks	Routine protection of a single critical section; the default type in the C++ standard library

Recursive mutex (recursive_mutex)	Allows the same thread to lock multiple times; needs the same number of unlocks	Scenarios where nested calls need to repeatedly access the same shared resource
Timed mutex (timed_mutex)	Supports non-blocking locking with a timeout	Avoids threads waiting indefinitely and prevents deadlock/hangs
System-level named mutex	Visible across processes; synchronizes threads in different processes	Scenarios where multiple processes share global resources, e.g., global synchronization on Windows

Basic usage conventions:

1. The standard flow follows the "lock → operate on shared resource → unlock" pattern. Directly calling lock()/unlock() manually can easily lead to forgetting to unlock due to exceptions or early returns, causing deadlocks.

2. Modern programming practice prefers RAII-style automatic lock management classes, such as C++'s std::lock_guard or Go's defer mu.Unlock(), which lock automatically at object construction and unlock at destruction, avoiding missed unlocks at the syntax level.

3. Key precautions:

Avoid deadlocks: when multiple threads acquire multiple locks, they must apply for locks in a globally consistent fixed order; circular waiting is forbidden.

Control lock granularity: only lock the critical-section code that truly needs protection; do not perform time-consuming I/O, network requests, etc. while holding the lock, to avoid greatly reducing program concurrency performance.

Never use mutex in interrupt context: locking a mutex may cause the thread to sleep, and interrupt context does not allow sleeping; otherwise the system will crash.

4In the Rust official documentation and standard library implementation, the thread-related Slot is not a single standalone concept, but rather an underlying storage unit in multithreading/concurrency scenarios, distributed across three core scenarios: channel implementations, the Tokio scheduler, and thread-local storage. All definitions come from the Rust standard library source code and official public documentation.

Message Slots in Channels

This is the most clearly defined Slot concept in the Rust standard library — the core storage unit of the mpsc channel's underlying implementation, defined directly as struct Slot<T> in the standard library source code.

Core structure: each Slot contains two fields:

msg: UnsafeCell<MaybeUninit<T>>: safely stores message data in an uninitialized state, avoiding unnecessary memory initialization overhead

state: AtomicUsize: an atomic status flag using three constants to mark the Slot lifecycle: WRITE(1) means the message has been written, READ(2) means the message has been read, and DESTROY(4) means the slot is being destroyed and recycled

Block-based management of unbounded channels: unbounded channels use a linked-list block structure, where each Block<T> contains BLOCK_CAP=31 Slots, allocating multiple messages to the same block consecutively, greatly reducing memory allocation frequency and improving cache hit rate

Lock-free synchronization logic: the sending thread marks the WRITE bit via atomic operations, and the receiving thread spins waiting for that flag before reading the data — concurrent-safe message passing without any mutex

4. True 50 Hz tick - this is the core.
   Every 20 ms: ticker.tick().await;
   And: MissedTickBehavior::Skip
   It is neither Burst nor Delay. Therefore, if one execution is too slow:

   tick 3 ← missed

   It will not:
   tick 3
   tick 3
   tick 3
   catch up.
   Nor will a single delay cause the entire cycle to permanently drift.tick 1tick 2tick 4

6.2 ROBOTD'S ONNX MODEL DECLARATION, INSTANTIATION, AND INVOCATION

MicroDuck does not have robotd directly manage all the details of ONNX Runtime. The actual ONNX model loading, session creation, shape validation, and inference have all been

---

LIFO Slots in the Tokio scheduler

An optimization component specific to Tokio's multithreaded async scheduler, explicitly documented as a scheduling feature in the official Tokio docs, used to optimize task latency in message-passing scenarios.

Core role: each worker thread maintains a dedicated Slot. When task A wakes task B by sending a message, task B is not pushed to the tail of the task queue, but stored directly in this Slot of the current thread; at the next scheduling, task B is prioritized, greatly reducing cross-thread latency in message passing

Characteristics and configuration: tasks in this Slot cannot be stolen by other worker threads; when a task's poll execution takes a long time, overall throughput may decrease. The official disable_lifo_slot() configuration method can turn off this optimization, pushing all tasks into the stealable local queue

Data Slots in Thread-Local Storage (TLS)

In the thread-local storage scenario, each individual thread has a dedicated storage location allocated for TLS variables — an underlying implementation detail of the Rust standard library std::thread::LocalKey and the third-party thread_local library.

Standard library implementation: LocalKey dynamically allocates a dedicated Slot for each thread via the indirect-jump thunk function to store TLS data, ensuring each thread accesses a completely independent copy without any synchronization operations

Third-party library implementation: the thread_local library uses a hierarchical atomic pointer bucket structure to assign a unique Slot index to each thread, achieving lock-free TLS data access with performance superior to the native pthread TLS interface

Lifecycle rules: the first time each thread accesses a TLS variable, the corresponding Slot is lazily initialized; when the thread exits, the data in the Slot is automatically dropped via Drop

extracted into duck-control::policy, i.e., inside the control loop; robotd is responsible for parameter parsing, policy selection, and the 50 Hz control loop.

The most core file: duck-control/src/policy.rs.

6.2.1 Define: define an ONNX model object

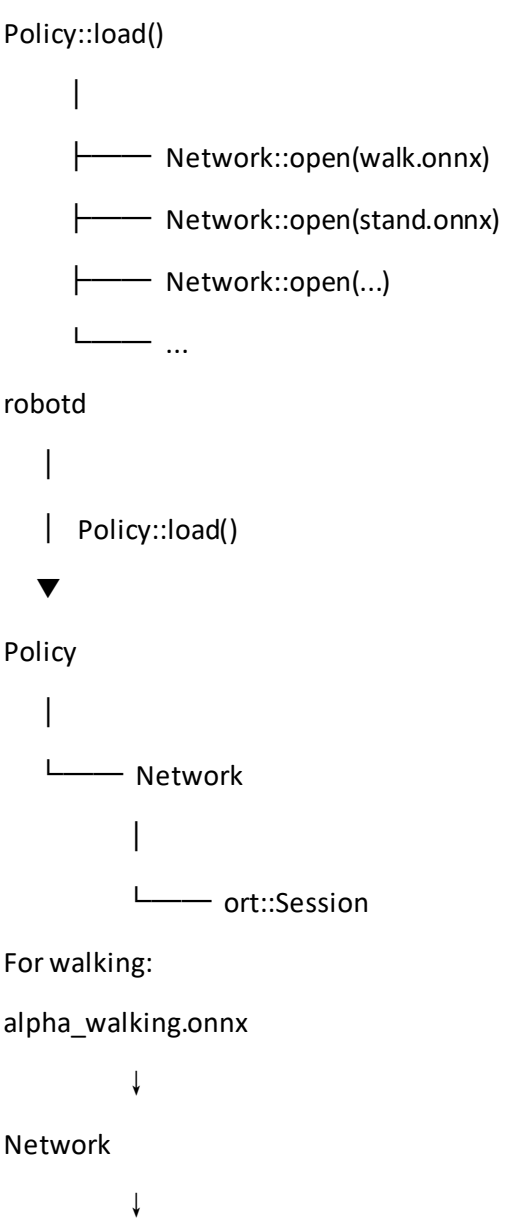
File: `duck-control/src/policy.rs`. The definition stage only declares "I have an ONNX model object". The `.onnx` file has not been loaded yet, and it cannot execute yet.

6.2.2 Create: load the `.onnx` file into a `Session`

Still in `duck-control/src/policy.rs`, creation happens inside `Network::open()`.

6.2.3 `robotd` creates it

Inside `Policy::load()`:



Session

6.2.4 Execute: call ONNX

The actual execution happens at:

`duck-control/src/policy.rs`

`Network::run()`.

The core can be simplified to:

let outputs = self.session.run(inputs)?;

This is the most critical line.

That is:

```text
observation
    |
    ▼
ONNX input tensor
    |
    ▼
self.session.run(...)
    |
    ▼
ONNX Runtime
    |
    ▼
[1, 14]
```

6.2.5 How does `robotd` call it?

In the control loop, `robotd` eventually reaches:

self.policy.infer(&observation, net)

Then:

Controller::step()

|

▼

Policy::infer()

|

▼

Network::run()

|

▼

session.run()

|

▼

ONNX

|

▼

14 actions

So you only need to remember this call chain:

robotd

|

▼

Policy::infer()

|

▼

Network::run()

|

▼

Session::run()

|

▼

alpha_walking.onnx

6.2.6 Summary of the three actions

① Define

Tells Rust:

```
struct Network {
    session: Session,
}
```

Meaning: a Network contains an ONNX Runtime Session.

② Create

```
let session = Session::builder()
    .commit_from_file(path)?;
```

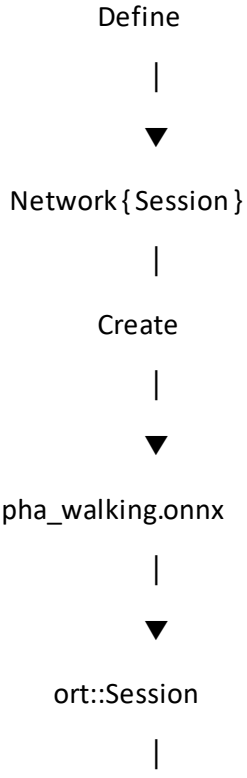
Meaning: load `alpha_walking.onnx` into ONNX Runtime and create a `Session`.

③ Execute

```
let outputs = self.session.run(inputs)?;
```

Meaning: feed the 61-dim observation to ONNX, run the neural network, and get the 14-dim action.

The simplest diagram





Define: declares what the object looks like.

Create = load the ONNX file to get a `Session`.**

Execute = `Session::run()`, input observation, output action.**

Once you clearly distinguish these three levels, the rest of `robotd`'s code will no longer confuse you.

6.3 FROM ONNX FILE TO PROGRAM OUTPUT: THE COMPLETE FLOW

This section talks about general ONNX concepts and workflow.

6.3.1 First understand: what an ONNX file itself is

An ONNX (Open Neural Network Exchange) file is essentially a "static computation graph description" — it only describes "how data flows and through which operators", and does not contain the execution engine itself. So a `.onnx` file "cannot run" by itself in a program; it must:

ONNX model file → Inference engine (Runtime) → Your program gets the output

This is exactly the reason for the layered design in the MicroDuck architecture: policy training is done in `microduck_rl` (PyTorch + PPO) (covered in the following sections); after exporting to ONNX, the `robotd` daemon on the MicroDuck robot loads it with ONNX Runtime and runs real-time inference at 50 Hz — the training framework and the inference engine are completely decoupled. The MicroDuck control loop is the closed loop of `state → observation → neural network (ONNX) → action → physics → new state`.

6.3.2 The complete process (four steps)

Step 1: Confirm the "model contract" (the most easily overlooked step)

The first thing when you get an ONNX file is not to write code, but to figure out its input/output contract — shapes, data types, semantics. Taking MicroDuck's walking policy as an example:

- Input: `obs`[5], float32, shape[6]`[1, 61]`

- Output: `actions`, float32, shape `[1, 14]` (corresponding to the actions of 14 servos)

- Control frequency: 50 Hz (inference once every 20 ms)

- Important detail: the observation normalizer is already baked into the ONNX graph, so callers should not normalize again.

How to inspect (choose any one):

```python
import onnx
model = onnx.load("policy.onnx")
print(onnx.helper.printable_graph(model.graph))
```

Or open it visually with the online tool netron.app.

Step 2: Install the inference engine

The most common choice is ONNX Runtime (maintained by Microsoft, supports Python/C++/C#/Rust/Java, runs with zero dependencies on CPU; there are also prebuilt packages for ARM64 boards like RK3566/RDK X5:

```bash
```

5obs is short for observation. It is one of the most common variable names in reinforcement learning, robotics, and autonomous driving. Many ONNX-exported reinforcement learning policy models name their input node obs. Taking a quadruped robot as an example, it is very common in Unitree, Isaac Lab, and Isaac Gym:

obs =[body angular velocity 3-dim

gravity direction 3-dim

12 joint positions

12 joint velocities

previous action

target velocity]

6In ONNX, Tensor, and deep learning, Shape is one of the most important concepts. It can be simply understood as:

Shape describes how large each dimension of a tensor is, similar to the dimensions of a matrix. For example: input: float32[1,3,224,224], where the shape is [1,3,224,224]. `obs`[ ], float32, shape[ ]`[1, 61]` means: the obs variable is a 61-dimensional vector of type float32.

pip install onnxruntime

If you have an NVIDIA GPU: pip install onnxruntime-gpu

```

Alternative engines (by scenario): TensorRT (NVIDIA acceleration), RKNN (Rockchip NPU), ONNX Runtime Web (browser — this is exactly what the official MicroDuck sandbox uses).

Step 3: Load the model and run one inference

Complete minimal Python example (corresponding to the MicroDuck contract):

```python

import numpy as np

import onnxruntime as ort

① Create the inference session (Session is a resource handle — create it once, don't put it in a loop)

sess = ort.InferenceSession("policy.onnx", providers=["CPUExecutionProvider"])

② Confirm inputs/outputs (cross-check against the model contract)

inp = sess.get_inputs()[0]

print(inp.name, inp.shape, inp.type)  # e.g.: obs ['1', 61] tensor(float)

③ Prepare input: build a float32 array strictly per the contract

obs = np.zeros((1, 61), dtype=np.float32)  # In practice, read the observation from IMU/servos

④ Inference: pass {input name: array}, get the output list

actions = sess.run(None, {inp.name: obs})[0]

print(actions.shape)  # (1, 14)

```

C++ equivalent (the real robot's `robotd` Rust logic is essentially the same):

```cpp

#include <onnxruntime_cxx_api.h>

Ort::Env env(ORT_LOGGING_LEVEL_WARNING, "demo");

Ort::Session session(env, L"policy.onnx", Ort::SessionOptions{});

```
std::vector<int64_t> dims{1, 61};
std::vector<float> obs(61, 0.0f);
auto mem = Ort::MemoryInfo::CreateCpu(OrtArenaAllocator, OrtMemTypeDefault);
Ort::Value input = Ort::Value::CreateTensor<float>(mem, obs.data(), obs.size(), dims.data(), dims.size());
auto outputs = session.Run(Ort::RunOptions{nullptr},
                           session.GetInputNames().data(), &input, 1,
                           session.GetOutputNames().data(), 1);
float* actions = outputs[0].GetTensorMutableData<float>();
```

In Rust you can use the `ort` crate; this is also the technical route used by the official MicroDuck stack.

Step 4: Embed into the control loop

A single `sess.run` gives you a raw tensor; in a robot program, you need to put it into a real-time loop:

```python
import time
RATE_HZ = 50                                # Must strictly match the control frequency used in training!
dt = 1.0 / RATE_HZ

while True:
    t0 = time.time()
    obs = read_sensors()                    # ① Compose the 61-dim observation from IMU/servo encoders
    actions = sess.run(None, {"obs": obs})[0]   # ② ONNX inference → 14-dim actions
    send_to_servos(actions[0])              # ③ Send actions to the servo bus
    time.sleep(max(0, dt - (time.time() - t0)))   # ④ Top up to 50 Hz
```

```

Engineering key points:

1. Frequency alignment: a policy trained at 50 Hz must also be called at 50 Hz during deployment, otherwise the temporal semantics of the observations are misaligned and the policy becomes unbalanced.

2. Pre/post-processing must match the model convention: for example, MicroDuck's normalization is already baked into the graph, so don't do it again; if it was not bake d during export, you must do it manually in code. The official exporter `scripts/export.py` automatically bakes the normalizer — this is required for correct behavior.

3. Create the session only once, and only call `run()` in the loop; the overhead of creating a new Session each time is unacceptable in real-time control.

# 7 MicroDuck Training

Address: https://github.com/pollen-robotics/microduck_rl

Getting started:

- README.md: Master the full "simulation → training → deployment" workflow (based on MuJoCo Warp simulation, PPO algorithm);
- src/agents/: Reinforcement learning agent implementation (reward design, policy network architecture);
- scripts/export.py: Script for exporting model to ONNX format (key link between simulation and hardware).

Core closed loop: Sim2Real pipeline (architecture design core)

This is the soul of Microduck's software architecture, and also the core value of open source:

1. Simulation training: Build robot model in MuJoCo Warp, train behavior policies through PPO algorithm (such as walking, fall recovery);
2. Model export: Export trained policy to ONNX format (microduck_rl/scripts/export.py);
3. Hardware deployment: Robot loads model via ONNX Runtime, executes in 50Hz control loop;
4. Iterative optimization: Observe differences between hardware and simulation through sensor data, adjust simulation parameters (such as friction, delay) and retrain.
```

CHAPTER 1 DOCUMENT DESCRIPTION AND TRAINING WORKFLOW OVERVIEW

1.1 Document Goal and How to Use

The goal of this document is: by reading only this document, you can complete the entire MicroDuck robot reinforcement-learning training workflow as described, including environment setup, task selection, smoke testing, formal training, training monitoring, recovery and warm start, result evaluation, ONNX export, publishing to Hugging Face, and real-robot installation. Every step provides directly copyable commands, parameter explanations, and expected results.

It is recommended to follow the chapters in order. The first four chapters (environment, tasks, smoke test) are mandatory steps before formal training and cannot be skipped; Chapter 5 is the core of training; Chapters 6-7 cover evaluation, export and publishing after training; Chapters 8-9 are for troubleshooting and quick command reference when problems occur.

1.2 Training Workflow Overview

Table 1 Training Workflow Overview

Step	Content	Key Command
1	Environment setup	git clone && uv sync
2	Select task	uv run list-envs
3	Smoke test	uv run train <TASK_ID> --env.scene.num-envs 64 --agent.max_iterations 5
4	Formal training	uv run train Mjlab-Velocity-Flat-MicroDuck --env.scene.num-envs 4096
5	Monitoring and recovery	wandb page / logs directory / resume
6	Result evaluation	uv run play / scripts/infer_policy.py
7	Export ONNX	uv run scripts/export.py <TASK_ID> --wandb-run-path <...>
8	Publish and deploy	uv run publish / robotctl policy

1.3 Key Concepts and Terminology

MicroDuck is a bipedal robot weighing about 800 g and about 25 cm tall, equipped with 14 Dynamixel XL330 servos. This repository provides its reinforcement-learning training environment; the trained policy is exported to ONNX and then loaded and executed by the runtime on the real robot.

mjlab: the training framework, based on MuJoCo Warp (GPU physics simulation) and PPO (rsl_rl implementation). The training control frequency is 50 Hz.

BAM actuator model: the key to sim2real. It models the XL330 with a voltage-control law, back-EMF, and Coulomb/Stribeck/load-dependent friction, and performs per-environment

domain randomization of battery voltage, load voltage drop, command latency and friction magnitude (FrictionDRBamActuator). Training and deployment use the same actuator model.

Observation/action contract: all policies share a 61-dim actor observation (48-dim proprioception + 13-dim command block [twist(3), head_pose(4), body_pose(6)]) and a 14-dim action; the ONNX export graph is [1,61]→[1,14]. Environments that do not use a command slot zero-fill that slot rather than deleting it.

Observation normalization: EmpiricalNormalization is used during training; at ONNX export, the normalization parameters are baked into the computation graph. Therefore deployment must use the ONNX exported by scripts/export.py; manually converted checkpoints are prohibited.

Backlash variant: every main task has a twin task with a ±1° (2° total) backlash model, which is closer to real servo transmission.

Domain randomization switches: listed as ENABLE_* boolean variables at the top of each task config file; they can be toggled as needed.

1.4 Repository and Related Projects

Repository address: https://github.com/pollen-robotics/microduck_rl , licensed under Apache-2.0 (3D model files under CC BY-SA-NC). Related projects include: microduck (robot project homepage, including the onboard runtime that runs exported policies), mjlab (training framework), and BAM (actuator model library by Rhoban).

CHAPTER 2 ENVIRONMENT SETUP

2.1 Hardware Requirements

Training must use a CUDA GPU, because training runs physics simulation on the GPU via MuJoCo Warp. The standard training configuration is 4096 parallel environments (--env.scene.num-envs 4096). If your machine has no GPU, you can submit training to Hugging Face Jobs in the cloud (see Section 5.7) without a local GPU.

On ARM boxes (e.g., DGX Spark / GB10, Jetson), the first uv sync needs to pull about 2 GB of CUDA wheels; uv's default 30-second HTTP timeout may interrupt the download. In that case, set UV_HTTP_TIMEOUT=600 first (see Section 2.4).

Note: 4096 parallel environments place high demands on GPU memory; the official recommendation is to use an NVIDIA GPU with large VRAM (e.g., RTX 40 series). For GPUs with around 6 GB VRAM (e.g., GTX 1660 Ti), first run smoke tests and training with 1024-2048 environments, confirm there is no out-of-memory (OOM), and only then increase gradually. The 64 environments used for smoke testing have very low VRAM requirements and can also run on CPU.

2.2 Software Requirements

The following software needs to be installed:

git (to clone the repository).

uv (Python package and project manager; all repository commands run through uv run).

Python 3.12 (pyproject.toml locks the Python version to >=3.12,<3.13; uv downloads and uses the matching version automatically, no manual install needed).

Installing uv: on macOS/Linux run curl -LsSf https://astral.sh/uv/install.sh | sh; on Windows run powershell -c "irm https://astral.sh/uv/install.ps1 | iex" in PowerShell. After installation, verify with uv --version.

The commands in this document are verified in Bash on Linux/macOS. Windows users are advised to run them in PowerShell (identical syntax) or WSL2; when GPU training is needed, make sure the driver and CUDA environment are usable by PyTorch.

When calling the wsl command through PowerShell on Windows, symbols such as double quotes, $HOME/$PATH variables, and && in the command line may be rewritten by the Windows layer, causing the command to fail or its output to be swallowed (exit code 0 but no output). The most reliable approach is to open the WSL terminal (Ubuntu window) directly and run the commands one by one, or write the commands into a shell script and run it at once.

2.3 Clone the Repository

Open a terminal and run the following commands to clone the repository and enter its directory:

```
git clone https://github.com/pollen-robotics/microduck_rl

cd microduck_rl
```

The repository contains the training environment source code (src/mjlab_microduck/), the robot MJCF model (including 3D-printed parts), scripts (scripts/), documentation (docs/, README.md, AGENTS.md) and tests (tests/).

2.4 Install Dependencies

Run uv sync in the repository root directory (all subsequent uv commands must run in the repository root) to install all dependencies (including PyTorch, mjlab, MuJoCo Warp, BAM, etc.):

```
uv sync
```

For the first sync on ARM boxes (DGX Spark / GB10, Jetson), set the timeout first:

```
UV_HTTP_TIMEOUT=600 uv sync
```

Dependency versions are locked by pyproject.toml. Note in particular: on linux-aarch64, the PyPI torch wheel is a CPU build; in pyproject.toml, [tool.uv.sources] routes torch to the cu129 index for aarch64 (x86_64 and HF Jobs still use PyPI). Do not remove the direct torch dependency or change the == pin to >=, otherwise the routing fails and a CPU build gets installed (see Chapter 8 for details).

The total dependency download is large (torch and CUDA-related libraries together about 6-8 GB); reserve at least 20 GB of disk space. If uv sync fails midway (common in network-constrained environments, with errors shaped like Request failed ... tunnel error / tcp connect

error / deadline has elapsed, and no .venv directory generated in the repository), it means the PyPI download channel is unstable; handle it in the following order:

(1) Configure a domestic mirror (recommended): create the user-level config file ~/.config/uv/uv.toml, write the following content, and rerun uv sync:

[[index]]

url = "https://pypi.tuna.tsinghua.edu.cn/simple"

default = true

(2) Or run UV_DEFAULT_INDEX=https://pypi.tuna.tsinghua.edu.cn/simple uv sync once (equivalent to the configuration above; it takes effect only for this command).

(3) If proxy environment variables (http_proxy/https_proxy, etc.) are set and the mirror still fails on direct connection, temporarily run unset http_proxy https_proxy HTTP_PROXY HTTPS_PROXY ALL_PROXY and then uv sync.

(4) After a uv sync failure, simply rerun to resume: already-downloaded packages are cached in the ~/.cache/uv/ directory; on repeated failures the failing package is usually different each time, so you can retry in a loop until it succeeds.

Running uv sync with a mirror rewrites the package download addresses in the repository's uv.lock to the mirror address (git status shows uv.lock modified), which is normal and does not affect subsequent training; keeping this modification lets later uv run and uv sync download directly from the mirror, faster and more stable.

2.5 Verify the Installation

Run the following commands to verify the environment is usable:

uv run list-envs

Expected output: a list of all training environments (tasks) registered by the current code. If the tasks are listed normally (see Chapter 3), the environment and dependencies were installed successfully.

Then check whether the GPU is available (it is normal for this check to be False when there is no GPU; you can switch to the Section 5.7 HF Jobs approach):

uv run python -c "import torch; print(torch.__version__, torch.cuda.is_available())"

You can also run uv run python -c "import mjlab_microduck" to confirm the project package imports.

To judge whether uv sync truly succeeded: a .venv directory should exist in the repository directory (you can use ls .venv/bin to view executables). If .venv does not exist or is empty, the dependencies were not fully installed; retry using the Section 2.4 methods. The first uv run may re-check the dependency index online; keep the network and mirror configuration consistent.

2.6 (Optional) Account Login

Training monitoring and publishing require the following accounts (all optional; local training can proceed without logging in; but publishing and HF Jobs are mandatory):

(1) wandb (Weights & Biases) - training monitoring platform. What it is: a machine-learning experiment tracking tool, register at https://wandb.ai (click Sign Up to register a free account), then run wandb login after registering (it will ask you to paste the API key the first time). What it is for: during training, metrics such as mean reward, episode length and each penalty term are synced in real time to the cloud dashboard, allowing you to plot curves, compare multiple runs, and view training progress remotely (see Section 5.3 on how to judge). When it is required: you can train locally without logging in (metrics are also written to the local logs/ directory); but to view curves in the browser, and when using --wandb-run-path for resume training, playback evaluation (play) or ONNX export (export.py), the corresponding wandb run record is needed, so login is recommended. Whether it costs money: free for personal and academic use (official Free plan is $0/month with unlimited personal projects; only enterprise commercial use is charged per seat); login is free, no payment required.

(2) Hugging Face - model hosting and cloud training platform. What it is: an AI model and dataset hosting platform, register at https://huggingface.co (click Sign Up to register a free account), then run hf auth login after registering, or set the HF_TOKEN environment variable. What it is for: first, when your machine has no GPU, submit training to run on Hugging Face Jobs cloud GPUs (append --hf-jobs to the training command, see Section 5.7); checkpoints are automatically uploaded back to your account. Second, after training, use uv run publish to publish the ONNX policy to HF Hub, and install it on the real robot with a one-click robotctl policy command (see Chapter 7). When it is required: login is mandatory only for publishing and HF Jobs; pure local training does not need it. Whether it costs money: registering, hosting public models and datasets are free (Pro subscription is an optional enhancement); but HF Jobs cloud training is billed per GPU hour, subject to the official hf.co/pricing; common tiers for reference: l4x1 (L4, 24 GB) about $0.8/hour, a10g-large (A10G, 24 GB) about $1.5/hour, a100-large (A100, 80 GB) about $2.5/hour; local training incurs no cost.

CHAPTER 3 SELECTING A TRAINING TASK

3.1 View the Task Registry

Use the output of uv run list-envs as the live registry (tasks change as the code is updated). The tables below come from the repository README and are for planning reference; the actual trainable tasks are determined by list-envs.

3.2 Task List

Table 2 Task List (from README Tasks)

Task ID	Terrain	Description
Mjlab-Velocity-{Flat,Rough}-MicroDuck	Flat/Rough	Main task: walking following a velocity command + head pose command

Task ID	Terrain	Description
Mjlab-VelStand-{Flat,Rough}-MicroDuck	Flat/Rough	Walking + fall recovery within a single policy
Mjlab-StandUp-{Flat,Rough}-MicroDuck	Flat/Rough	Stand up from face-down/face-up/sitting posture, maintain standing, and support body pose control
Mjlab-SitStand-{Flat,Rough}-MicroDuck	Flat/Rough	Controlled sit ↔ stand transitions (gentle actions), head can command
Mjlab-GroundPick-{Flat,Rough}-MicroDuck	Flat/Rough	Crouch to touch the ground with the beak tip, then return to standing
Mjlab-BallKick-Flat-MicroDuck	Flat	Kick a 70 mm/15 g ball forward (actor cannot see the ball)
Mjlab-Roulade-Flat-MicroDuck	Flat	Forward roll, landing back on both feet
Mjlab-Velocity-Flat-MicroDuck-Rollers	Flat	Roller speed tracking (passive wheels under the feet)
Mjlab-Velocity-Swizzle-MicroDuck	Flat	Classic symmetric swizzle gliding
Mjlab-RollerCrouch-Flat-MicroDuck	Flat	Crouch while roller-gliding
Mjlab-RollerSlope-Flat-MicroDuck	Slope	Roller downhill gliding
Mjlab-RollerStandUp-Flat-MicroDuck	Flat	Stand up from the ground onto the rollers
Mjlab-Spin-Flat-MicroDuck	Flat	Rapid in-place rotation on rollers

Beginners are advised to start with the main task Mjlab-Velocity-Flat-MicroDuck: the README quick start uses it as an example, its training difficulty and duration are moderate, and it can serve as a starting point for other tasks.

3.3 Task Naming Convention and Backlash Variants

Task IDs have the form Mjlab-<task>-<terrain>-MicroDuck, e.g., Mjlab-Velocity-Flat-MicroDuck, Mjlab-StandUp-Rough-MicroDuck.

Each main task has a Backlash twin task: insert -Backlash before MicroDuck in the task ID, e.g., Mjlab-Velocity-Flat-Backlash-MicroDuck. The Backlash model adds ±1° backlash (2° total) in series at the 14 servo joints. Backlash is modeled correctly: each servo has an undriven passive_<joint>_backlash hinge, and the real encoder sits on the output side of the backlash, so the firmware PD simulation and the joint_pos/joint_vel observations both read "through" the backlash. Observation and action dimensions are unchanged; no ONNX-export or runtime changes are needed. Backlash policies are usually closer to real servo behavior; for the strongest sim2real, train the Backlash variant.

3.4 Robot and Observation/Action Contract

The robot model is located at src/mjlab_microduck/robot/microduck/ and is exported from Onshape via onshape-to-robot. Different tasks use different collision models, as shown in the table below.

Table 3 Robot Models and Uses

XML Model	Use
robot_walk.xml	Velocity (strips torso/head collisions - low fall cost)
robot_groundcontact.xml	VelStand, StandUp, SitStand, GroundPick, BallKick, Roulade (curated ground-contact collision set; the body can truly touch the ground)
robot_groundcontact_rollers.xml	Roller tasks (passive wheels)
robot_allcollisions.xml	Full collision model (every part has a collision geometry); currently used by no task
robot_*_backlash.xml	Backlash variants (generated by add_backlash.py)

Joint layout (14 servos): 0-4 left leg (hip_yaw, hip_roll, hip_pitch, knee, ankle), 5-8 neck/head (neck_pitch, head_pitch, head_yaw, head_roll), 9-13 right leg. Undriven joints are uniformly named with a passive_ prefix (e.g., roller passive wheels, backlash hinges); actuators, joint observations and pose rewards select only servo joints via the regex ^(?!passive_).*.

Observation contract (shared by all policies): 61-dim actor observation = 48-dim proprioception + 13-dim command block [twist(3), head_pose(4), body_pose(6)], in that order. Environments that do not use a command slot zero-fill that slot (keep the observation item, sample a very small range); never delete the slot. The action dimension is 14.

CHAPTER 4 SMOKE TESTING

4.1 Why Smoke Test First

AGENTS.md explicitly requires that a smoke test be run before any long training. A smoke test with 64 environments and 5 iterations takes only a few minutes and can catch about 95% of configuration errors (environment construction, observation dimensions, model loading, BAM actuator, rendering, etc.). Skipping the smoke test and going straight to long training means errors only surface hours later, at a very high cost.

4.2 Run the Smoke Test

Taking the main task as an example, run:

uv run train Mjlab-Velocity-Flat-MicroDuck --env.scene.num-envs 64 --agent.max_iterations 5

Expected result: the environment builds successfully; the log shows observation dimension 61 and action dimension 14; training progresses normally through 5 iterations and ends

normally; no NaN; a record of this run appears under the logs/ directory. The <TASK_ID> in the command can be replaced with any task ID from list-envs.

4.3 Troubleshooting Failed Smoke Tests

If the smoke test fails, common causes and solutions:

Wrong or unregistered task name: run uv run list-envs to check the task ID spelling against the current registry.

CUDA/environment errors: verify the GPU driver, CUDA, and the GPU build of torch (check per Section 2.5).

Observation/action dimension mismatch: check whether a command slot or the height_scan observation item was accidentally removed (the velocity config already removed height_scan; other configs should follow the same practice).

NaN appears: mostly a sign error in a reward penalty term (see Section 5.3 and Chapter 8).

CHAPTER 5 FORMAL TRAINING

5.1 Start Training

Taking the main task as an example, the standard training command (4096 parallel environments):

uv run train Mjlab-Velocity-Flat-MicroDuck --env.scene.num-envs 4096

The README notes: with 4096 environments, a usable gait can be obtained in about 1-2 hours (depending on GPU performance). During training, reward metrics are printed each iteration and synced to wandb; checkpoints are saved to the local logs directory at fixed intervals (see Section 5.3).

5.2 Training Parameter Description

Table 4 Common Training Parameters

Parameter/Item	Description
--env.scene.num-envs	Number of parallel environments. Standard config 4096; reduce when VRAM is limited (e.g., 2048/1024); smoke test uses 64.
--agent.max_iterations	Maximum number of iterations, default 50000, more than enough for common task budgets (see Section 5.4).
--agent.run-name	Name of this run (used to distinguish multiple runs, e.g., resume). Defaults to the same as experiment_name.
--agent.load-checkpoint	Checkpoint file name to load, e.g., model_29999.pt, used together with --agent.resume.

Parameter/Item	Description
--agent.resume	When set to True, resumes training from the specified checkpoint (restoring weights, observation normalizer, and optimizer state).
--wandb-run-path	wandb run path in the format entity/project/run_id. Used when resuming training or evaluating/exporting based on a run.
--wandb-checkpoint-name	Specifies the checkpoint file name within a wandb run (used with --wandb-run-path).
--hf-jobs	Append to any train command to submit training to run on Hugging Face Jobs in the cloud (see Section 5.7).

Training hyperparameters and domain-randomization configuration are located in the task config file src/mjlab_microduck/tasks/microduck_velocity_env_cfg.py: the ENABLE_* domain-randomization switches are at the top of the file; MicroduckRlCfg at the end holds the PPO hyperparameters. Default values for the main walking recipe (velocity task): actor/critic networks are (512,256,128) hidden layers with elu activation and observation normalization enabled; Gaussian distribution with initial std 1.0; PPO learning rate 1e-3 (adaptive scheduling), value_loss_coef 1.0, clip_param 0.2, entropy_coef 0.01, 5 learning epochs per round, 4 mini-batches, gamma 0.99, lam 0.95, desired_kl 0.01, max_grad_norm 1.0; save_interval 250 (a checkpoint every 250 iterations), num_steps_per_env 24, max_iterations 50000. The wandb project name is mjlab_microduck and experiment_name is velocity (i.e., the log directory name).

5.3 Training Monitoring and Result Judgment

Training metrics are synced in real time to the wandb project mjlab_microduck (run names look like <timestamp>_velocity). Local logs are under the logs/<experiment_name>/ directory (logs/velocity/ for the velocity task); checkpoints are saved as model_<iteration>.pt, e.g., model_250.pt, model_500.pt, one generated every save_interval (default 250) iterations.

To judge whether training is normal, observe the following per iteration:

Mean reward keeps increasing.

Episode length matches the task expectation (e.g., a walking task should not end prematurely).

Each penalty term satisfies Episode_Reward/<penalty> $\leqslant$ 0 (a penalty is a self-negating value multiplied by a positive weight; if the weight is negative, it creates double negation that turns violations into rewards, and training will necessarily produce NaN or malformed behavior).

The main task term truly grows: total reward may rise only through regularization while the skill never happens; confirm the task's core term is growing.

Note: Episode_Reward/<term> records a weighted value - an item with weight 0 is always 0 regardless of behavior, so interpret it together with the weight schedule.

After training ends or stops early, select a checkpoint under the logs/velocity/ directory. From experience: usable checkpoints for simple skill tasks often appear near iteration 1000; for gait-type tasks, near iterations 4000-6000 (see Section 5.4).

5.4 Training Time Budget

Budget given by AGENTS.md (4096 environments):

Simple episodic skills (e.g., Roulade, SitStand): about 1000 iterations.

Gait and curriculum-recovery types (e.g., Velocity, VelStand): 4000-6000 iterations.

max_iterations defaults to 50000, more than enough to cover the above budgets. You can end early once the goal is reached (terminate with Ctrl+C; already-saved checkpoints are unaffected); or pre-limit the iteration count with --agent.max_iterations.

5.5 Resume Training from a Checkpoint

If training is interrupted or you want to continue, use --agent.resume to resume from a checkpoint (also restoring weights, observation normalizer, and optimizer state):

uv run train Mjlab-Velocity-Flat-MicroDuck --env.scene.num-envs 4096 \

--agent.run-name resume --agent.load-checkpoint model_29999.pt --agent.resume True

Checkpoint files are under the logs/velocity/ directory; to resume from wandb, use --wandb-run-path <entity/project/run_id> and --wandb-checkpoint-name model_XXXX.pt.

5.6 Cross-Task Warm Start

Warm start differs from resume: mjlab's runner stores common_step_counter in the checkpoint and restores it on load (along with the iteration number). Therefore directly loading another task's checkpoint makes all step-based curricula jump to the final stage at the very first iteration. If you want to initialize a new task with another task's checkpoint and have the new task's curricula run again from stage 0, you must prefix the command with the environment variable MICRODUCK_WARM_START=1:

MICRODUCK_WARM_START=1 uv run train Mjlab-VelStand-Rough-Backlash-MicroDuck --env.scene.num-envs 4096 \

--agent.resume True --wandb-run-path pollen-robotics/mjlab_microduck/fhathosb \

--wandb-checkpoint-name model_3750.pt --agent.max_iterations 1500

Warm start keeps the weights, observation normalizer and optimizer, while zeroing common_step_counter and the iteration number so the curricula run again from stage 0. At the same time, in the target config you should fold the source task's inherited curricula into the final stage (see the _collapse_curricula_to_final implementation in the velstand task), because the loaded policy was trained under exactly those conditions.

Note: warm start only between tasks whose observation normalizers are compatible. For example, the stand-up expert's twist std is 0.005; it must never be used to initialize a walking environment.

5.7 No-GPU Option: Hugging Face Jobs

When your machine has no GPU, append --hf-jobs to any train command to submit training to run on Hugging Face Jobs in the cloud (see the repository's scripts/hf/README.md for details).

One-time preparation (only needed once):

Run hf auth login to log in to Hugging Face (or set the HF_TOKEN environment variable).

Run wandb login to log in to wandb (HF Jobs training is also recorded to the wandb project mjlab_microduck).

Submit training (using the main task as an example):

uv run train Mjlab-Velocity-Flat-MicroDuck --env.scene.num-envs 4096 --hf-jobs

Table 5 HF Jobs Common Parameters

Parameter/Item	Description
--namespace	HF namespace (your username or organization name). Interactive selection when omitted; explicit passing is recommended for automation.
--flavor	Hardware spec: l4x1 (default), a10g-large, a100-large.
--timeout	Maximum task duration, default 12h.
--detach	Return immediately after submission without following the log output.
--dry-run	Only print the submission plan, without actually submitting.
--run-name	Name of this cloud run (determines the checkpoint repository name).
--no-uv-cache	Do not use the shared uv cache bucket (default uses <namespace>/mjlab-uv-cache).
--no-wandb	Do not write to wandb for cloud training.

Mechanism description: at submission, the local source is packaged into a tarball and uploaded to a private dataset repository; the task runs uv sync inside the cloud container (keeping pyproject.toml consistent to ensure remote reproducibility). During training, checkpoints are uploaded by uploader.py every 60 seconds to the HF model repository <namespace>/<run-name>, so you can browse and download model_XXXX.pt files directly on the Hugging Face page. After successful submission, the terminal prints the job id, log address, and checkpoint repository link.

Task management (in Python): you can use huggingface_hub's HfApi to manage jobs, e.g., HfApi().list_jobs(namespace=...), fetch_job_logs(job_id=..., namespace=...) to view logs, and cancel_job(job_id=..., namespace=...) to cancel a job.

CHAPTER 6 EVALUATING TRAINING RESULTS

6.1 Viewer Playback (uv run play)

Play back a policy from a wandb run in the viewer (load the checkpoint and watch it in MuJoCo simulation):

```
uv run play Mjlab-Velocity-Flat-MicroDuck --wandb-run-path <entity/project/run_id>
```

Replace <entity/project/run_id> with the entity/project/run ID from wandb.

6.2 Deployment Rehearsal (infer_policy.py)

scripts/infer_policy.py loads the exported ONNX policy in CPU MuJoCo and drives it in real time with the keyboard, simulating the real runtime's policy hot-switching (walking/standing/skills switch freely). This is the most important rehearsal tool before deploying to the real robot. Multiple policies can be loaded at once:

```
uv run scripts/infer_policy.py --walking walk.onnx --standing stand.onnx \
    --sitstand sitstand.onnx --roulade roulade.onnx --new-cmd-obs
```

Table 6 infer_policy.py Keyboard Controls

Key	Action
W/S	Forward/backward linear velocity
A/D	Left/right strafe linear velocity
Q/E	In-place turning angular velocity
G	Trigger the ground-pick policy
Y	Sit ↔ stand transition (sitstand policy)
R	Trigger the forward-roll policy
K/L	Trigger the left-foot/right-foot kick policy

Table 7 infer_policy.py Common Parameters

Parameter/Item	Description
--walking/--standing/--sitstand/--ground-pick/--kick-left/--kick-right/--roulade/--slope	Specify the ONNX file path for each policy; provide at least one of walking, standing, or sitstand.
--new-cmd-obs	Use the unified 13-dim command observation layout (61 dims). sitstand, kick, and roulade policies must add this parameter.
--debug	Print observations and actions frame by frame, for comparison.
--save-csv	Save observations and actions to a CSV file, for sim2real comparison.

Parameter/Item	Description
--record	Enable recording: on Ctrl+C, save the observations as a pickle file.
--no-bam	Fall back to the XML position actuator (not the actuator used in training; only for comparison, generally not for evaluation).
--vin	BAM battery voltage (default 7.4 V). Training samples voltage per environment; here it is pinned to one value.
--vin-drop-gain	Load-dependent voltage-drop gain (default 0.1 V/Nm), corresponding to the training DR range.
--kp-fw	BAM firmware P gain (default same as the training value).
--roller	Use the roller robot model (robot_walk_rollers.xml).
--scene	Specify a scene XML path, overriding the default scene selection.

Important: infer_policy.py uses the same BAM M6 XL330 model as training for the servos (voltage control + load-dependent friction, via bam.mujoco.MujocoController); --vin, --vin-drop-gain, and --kp-fw pin the training domain-randomization range to a deterministic value so it can be compared with the real robot.

6.3 Regression Tests

The repository ships configuration-invariance and reward-function regression tests (they run on CPU, no GPU needed), locking down the joint-index mapping, reward-sign conventions, and NaN protection. After modifying a config, it is recommended to run:

```
uv run --with pytest pytest tests/
```

CHAPTER 7 EXPORT ONNX AND PUBLISH

7.1 Export ONNX

Export a trained checkpoint to deployment ONNX:

```
uv run scripts/export.py Mjlab-Velocity-Flat-MicroDuck --wandb-run-path <...>
```

The optional parameter --checkpoint N specifies the checkpoint iteration number within the run (e.g., 3000); if not specified, the default selection is used. The export produces output.onnx (an output path can be specified), and the computation graph has shape [1,61]→[1,14].

Key point: the exporter bakes the observation normalizer (EmpiricalNormalization) into the ONNX graph. Deployment must use the ONNX exported by scripts/export.py - a manually converted checkpoint's model sees unnormalized observations at runtime and behaves abnormally.

7.2 Verify the Exported File

Load the exported ONNX with infer_policy.py to confirm it can infer normally:

uv run scripts/infer_policy.py --walking output.onnx

After confirming the policy responds to keyboard commands and the actions are reasonable, proceed to the publishing step.

7.3 Publish to Hugging Face Hub

uv run publish publishes the policy to Hugging Face Hub in a form loadable by the robot daemon: a policy.onnx with observation normalization, a manifest.json following schema 2, and a README explaining how to run it. Anyone who owns a microduck can install it with one click, without publishing a new daemon version.

Export and publish from a wandb run (export through the unique safe path, then upload):

uv run publish --task Mjlab-PoliteBow-Flat-MicroDuck \

--wandb-run-path <entity/project/run_id> --checkpoint 3000 \

--repo <user>/microduck-polite-bow --kind episodic --duration-s 4.0 \

--description "Bows from a two-foot stand and comes back up."

Publish from an already-exported ONNX (already verified; no repeated export):

uv run publish --onnx output.onnx --repo <user>/microduck-flamingo --kind perpetual --unwind-s 1.5 --twist-help "[flag, side, 0]"

Publish a new walking policy for the gait slot:

uv run publish --onnx output.onnx --repo <user>/microduck-my-walk --kind perpetual --slot walk

Remix/fine-tuned policy (name the parent policy when basing on someone else's published policy; the model card records base_model and establishes a Hub link):

uv run publish --onnx output.onnx --repo <user>/microduck-deep-bow --kind episodic --duration-s 5.0 --base-model <owner>/microduck-polite-bow

Rehearsal (does not touch the Hub; only prints what will be uploaded):

uv run publish --onnx output.onnx --repo <user>/microduck-bow --kind episodic --duration-s 4.0 --dry-run

Table 8 The Two --kind Forms and Required Parameters

Form	Parameters	Description
episodic	--duration-s (--chain optional)	Runs for --duration-s seconds then automatically returns to the standing pose (one-shot skills such as kicking, forward roll, bowing); with --chain, holding the button repeats it
perpetual (gait)	--slot walk (or stand)	A new walking/standing gait; the installer loads it with robotctl policy load walk <repo>
perpetual (hold pose)	--unwind-s (--idle optional)	E.g., one-leg stand: the daemon first drives the idle twist (--idle, default zero) for --unwind-s seconds, then hands back the gait to avoid suddenly letting go on one foot; the installer runs it as a one-shot policy with policy add ... --hold <seconds>

Automatic checks before publishing: verify the computation graph is [1,61] → [1,14] (51-dim legacy policies are rejected); run on reasonable inputs and reject NaN or constant outputs; fill the training block from git and wandb (task, commit, branch, dirty flag, run, checkpoint); reject overwriting when a same-named .onnx already exists in the repository (requires --force). Repositories are created private by default; --no-private makes them public, and --tag v1 tags a version.

Note: only constant-command policies can be published this way. Phase-driven actions (e.g., ground pick) and the pose flags sit ↔ stand are driven by the daemon itself and belong to the official collection pollen-robotics/microduck-policies.

7.4 Real-Robot Installation (robotctl)

Install the published policy on the real robot (microduck runtime, requires root privileges):

sudo robotctl policy add polite-bow <user>/microduck-polite-bow # episodic: duration taken from manifest

sudo robotctl policy add flamingo <user>/microduck-flamingo --hold 5 # hold pose: specify the duration yourself

sudo robotctl policy load walk <user>/microduck-my-walk # gait: load into the walk slot

robotctl robot do polite-bow

7.5 Render a Playback Video (Optional)

You can render the policy's playback in simulation to a video (the Hub's playback component can play it after upload):

uv run scripts/render_policy.py output.onnx --seconds 10 --video media/bow.mp4

Then attach the video at publish time: uv run publish --onnx output.onnx --repo <user>/microduck-bow --kind episodic --duration-s 4.0 --video media/bow.mp4.

CHAPTER 8 COMMON PROBLEM TROUBLESHOOTING

Table 9 Common Problems and Solutions

Symptom	Possible Cause	Solution
uv sync fails with a mid-way timeout	Slow network, or first pull of about 2 GB of CUDA wheels on an ARM box exceeds uv's default 30-second HTTP timeout	For the first sync set UV_HTTP_TIMEOUT=600 uv sync
torch.cuda.is_available() is False, or training reports a select_gpus IndexError	The CPU-only torch from PyPI got installed on aarch64	Confirm that torch is a direct dependency and is == pinned in pyproject.toml ([tool.uv.sources] applies to direct dependencies only); rerun uv sync
NaN during training/smoke test	Sign error in a reward penalty: a self-negating penalty used a negative weight, making a double negation into a reward	Check the weight signs in the task config; every Episode_Reward/<penalty> must be ≤0
Checkpoint not found	Wrong wandb run path or iteration number; wrong local path	Verify --wandb-run-path is entity/project/run_id and --checkpoint is a numeric iteration number; local checkpoints are at logs/<experiment_name>/model_XXXX.pt
Abnormal behavior after deploying a manually converted checkpoint	Observations were not normalized at deployment	Must export ONNX with scripts/export.py (the normalizer is baked into the graph); manual conversion is prohibited
Curricula jump to the final stage after resuming training	Loaded another task's checkpoint without warm start	Continue the same task with normal resume; for cross-task inheritance use MICRODUCK_WARM_START=1 (and fold the source curricula)
play works but the real robot behaves abnormally	sim2real is not aligned	Rehearse with infer_policy.py (BAM consistent with training); pin the training DR range with --vin/--vin-drop-gain/--kp-fw; confirm the command observation layout (--new-cmd-obs) is consistent

Symptom	Possible Cause	Solution
Observation dimension is not 61	A command slot was accidentally removed or height_scan was handled inconsistently	Keep the 13-dim command slots (zero-fill when unused); the velocity config removed height_scan; follow the same practice in other configs
uv sync repeatedly fails to download (tunnel error / tcp connect error / deadline has elapsed)	Constrained network: PyPI direct connection or proxy tunnel is unstable	Configure the Tsinghua mirror and retry (see Section 2.4); if needed, first run unset http_proxy https_proxy HTTP_PROXY HTTPS_PROXY ALL_PROXY; after a failure, rerun to resume from the cache

In addition, a few lessons from training practice: if a wandb metric happens to drop at a curriculum-stage boundary, the curriculum pacing is wrong (stages should be lengthened or introduced later, never earlier); reverse-forward curricula (generating episodes from mid-action) is a reliable way to solve "learns only the beginning, never the last step." When a run "fails," first do a headless evaluation on the actual checkpoint before changing the rewards - many past "failures" turned out to be premature checkpoints, success criteria that split one behavior cluster in two, or reward caps conflicting with measured physics. Judge by the actual rollout performance (e.g., "it can roll but lands face-down one time in three"), and let the user decide when it is good enough.

CHAPTER 9 QUICK COMMAND REFERENCE

Table 10 Quick Command Reference

Purpose	Command
View the task registry	uv run list-envs
Smoke test	uv run train <TASK_ID> --env.scene.num-envs 64 --agent.max_iterations 5
Formal training	uv run train Mjlab-Velocity-Flat-MicroDuck --env.scene.num-envs 4096
Cloud training (no GPU)	uv run train <TASK_ID> --env.scene.num-envs 4096 --hf-jobs
Resume training	uv run train Mjlab-Velocity-Flat-MicroDuck --env.scene.num-envs 4096 --agent.run-name resume --agent.load-checkpoint model_XXXX.pt --agent.resume True
Cross-task warm start	MICRODUCK_WARM_START=1 uv run train <TASK_ID> --env.scene.num-envs 4096 --agent.resume True --wandb-run-path <entity/project/run_id> --wandb-checkpoint-name model_XXXX.pt
Viewer playback	uv run play <TASK_ID> --wandb-run-path <entity/project/run_id>

Purpose	Command
Deployment rehearsal	uv run scripts/infer_policy.py --walking output.onnx --new-cmd-obs
Export ONNX	uv run scripts/export.py<TASK_ID> --wandb-run-path <...>
Render playback video	uv run scripts/render_policy.py output.onnx --seconds 10 --video media/out.mp4
Publish policy	uv run publish --task <TASK_ID> --wandb-run-path <...> --checkpoint N --repo <user>/microduck-<name> --kind episodic --duration-s 4.0
Regression tests	uv run --with pytest pytest tests/

APPENDIX A KEY REPOSITORY FILES AND FURTHER READING

The following files are the entry points for a deeper understanding of the training configuration and experience:

Table 11 Key Repository Files

File	Content
README.md	Quick start, task list, Backlash, actuator model, robot model, publishing notes
AGENTS.md	Training experience and rules (reward-design sign conventions, how to read a run, sim2real pitfalls, budgets)
src/mjlab_microduck/tasks/microduck_velocity_env_cfg.py	Main walking recipe: domain randomization, observations, commands, PPO hyperparameters (other task configs are based on it)
src/mjlab_microduck/tasks/mdp.py	All custom reward, event, observation, command, and curriculum functions
src/mjlab_microduck/tasks/backlash.py	Backlash variant environment wrapper
src/mjlab_microduck/actuator/friction_dr_bam.py	BAM actuator + friction domain randomization + backlash encoder feedback
src/mjlab_microduck/export.py	ONNX export (normalizer baked in)
src/mjlab_microduck/publish/	publish command: manifest construction, ONNX validation, Hub upload
scripts/hf/README.md	Hugging Face Jobs submission and parameter description

File	Content
docs/velstand_policy.md, docs/sitstand_policy.md	Real policy training history and warm start examples
tests/	Configuration-invariance and reward-function regression tests

Related projects: microduck (robot project homepage and onboard runtime, https://github.com/pollen-robotics/microduck), mjlab (training framework, https://github.com/mujocolab/mjlab), BAM (actuator model library, https://github.com/Rhoban/bam).

8 BROWSER SIMULATION TOOL (VISUAL UNDERSTANDING OF ARCHITECTURE OPERATION)

No local deployment needed, experience the architecture closed loop directly in the browser:

Address: https://huggingface.co/spaces/pollen-robotics/microduck-simulator

Getting started:

- Operation interaction: Use WASD / gamepad to control the robot (walking, kicking, roller skating), intuitively experiencing the "input command → policy inference → motor execution" pipeline;
- Core observation: Switch between "leg / roller skating mode" (press M key), understand how different ONNX policies are loaded and executed;
- Debug mode: Add URL parameter ?boot=1 to enter BIOS console, view real-time logs of model loading and policy initialization (corresponding to the hardware startup process).

9 OFFICIAL DOCUMENTATION AND FEATURE DESCRIPTION (SUPPLEMENTARY ARCHITECTURE DESIGN PHILOSOPHY)

- Start from the docs/ directory in the GitHub repository, focus on agents.md (reinforcement learning policy design manual);
- Refer to the product page technical description: https://pollen-robotics.com/microduck/ (clarifying the interaction boundary between software and hardware, such as sensor data format, motor control protocol).

10 RL ALGORITHMS

10.1 WHAT PROBLEM DOES REINFORCEMENT LEARNING ACTUALLY SOLVE?

Cumulative return:$G_t = R_{t+1} + \gamma R_{t+1} + \gamma^2 R_{t+3} + \ldots\ldots$

That is:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \qquad \textit{Formula 1}$$

where:

- R_{t+k+1}: the immediate reward obtained after executing an action
- $\gamma \in [0,1]$: the discount factor
- G_t: the total return starting from time t.

Physical meaning of the formula: from Formula 1, the cumulative return is the sum of the returns of all subsequent actions starting from the present moment. More intuitively:

Time t is just a starting point. We choose an action at the current moment, obtain a reward after reaching the next state, then execute the next action to obtain A_t, then the next step to obtain R_{t+1}, and continue accumulatingR_{t+2}, then obtain in the next step R_{t+3}.

Therefore it can be understood as: the sum of all future rewards. Only because the further the future, the more uncertain, reinforcement learning usually multiplies by a discount factor γ to make the influence of distant rewards smaller. G_t =The sum of all future rewards. Only because the further the future, the more uncertain, reinforcement learning usually multiplies by a discount factor γ so the influence of distant rewards becomes smaller.

Reinforcement learning does not compare the reward of a single step, but the cumulative return corresponding to each of multiple paths, and then chooses the policy with the G_t and then choose the $G_t$larger one.

So what reinforcement learning truly optimizes is not:

"Maximize the next-step reward."

but rather:

"From now on, maximize the reward accumulated over the long term."

This is one of the most fundamental differences between reinforcement learning and many ordinary supervised learning problems.

10.2 WHY DO WE NEED "STATE, ACTION, REWARD"?

Reinforcement learning usually describes the environment with a mathematical object:

$$\mathrm{MDP} = (\mathrm{S}, \mathrm{A}, \mathrm{P}, \mathrm{R}, \gamma)$$

where:

- S: state space

- A: action space

- P: state transition probability

- R: reward function

- γ: the discount factor

State transition probability $\mathrm{P}(S'|\mathrm{s}, \mathrm{a})$ Formula 2

Formula 2 expresses the probability that, currently in state s, after taking action a, the next state becomes s'.

For example, $\mathrm{P}(S'_{左}|\mathrm{s}_{中}, \mathrm{a}_{左}) = 0.9$ means: currently in the middle position, choosing to go left, with 90% probability it actually reaches the left.

Physical meaning: this is the dynamics law of the environment.

For example, for a robot: current position + control force ⟶ next position, but the real world has noise, so it is not deterministic. For example, the robot wants to go forward; although the actions taken are all forward, in reality it may go forward, left, or right (due to mechanical structure errors, control errors, and other factors):

$$\mathrm{P}\left(\text{向前}\middle|\mathrm{s}, a_{\text{前进}}\right) = 0.95$$

$$\mathrm{P}\left(\text{向左}\middle|\mathrm{s}, a_{\text{前进}}\right) = 0.03$$

$$\mathrm{P}\left(\text{向右}\middle|\mathrm{s}, a_{\text{前进}}\right) = 0.02$$

Therefore reinforcement learning is essentially making decisions in a dynamic system that may contain randomness.

10.3 What is a policy?

Policy: $\pi(\mathrm{a}|\mathrm{s})$ Formula 3

Mathematical meaning:

$$\pi(\mathrm{a}|\mathrm{s}) = \mathrm{P}(A_t = \mathrm{a}|S_t = \mathrm{s})$$

It expresses: given the current state s, the probability of choosing action a.

For example:

$$\pi(\mathrm{a}_{左}|\mathrm{s}) = 0.8$$

$$\pi\left(\mathrm{a}_{右}\middle|\mathrm{s}\right) = 0.2$$

It says that in this state the robot: has an 80% probability of taking the left action, and a 20% probability of taking the right action.

Physical meaning: a policy is the robot's behavior rule; Formula 3 expresses what action to take in the current state. It can also be understood as a controller:

$$s \longrightarrow a$$

For example: seeing an obstacle $\longrightarrow$ turn left.

One of the core tasks of reinforcement learning is: finding the optimal policy.

10.4 Why do we need a Value Function?

We already know that $\boldsymbol{G_t}$ denotes how much reward we can ultimately obtain starting from now.

But here is the problem: if we are currently in state s and the future has randomness, then this $\boldsymbol{G_t}$ may differ every time. So we consider its mathematical expectation.

$$V^{\pi}(s) = E_{\pi}[G_t|S_t = s] \qquad \text{Formula 4}$$

Its meaning is: starting from state s and acting according to the policy π (note: the policy, not an action), the average value of the future cumulative return. Note that this is very important:

$$V^{\pi}(s) \neq G_t$$

instead of

$$V^{\pi}(s) = E[G_t]$$

G_t is the return from one concrete experience, while $V^{\pi}(s)$ is the average return over many possible experiences.

Physical meaning: you can understand $V^{\pi}(s)$ as: "how much is this position (state) worth?"

For example:

State	$V^{\pi}(s)$
State 1 (one step from the goal)	+9
State 2 (three steps from the goal)	+6
State 3 (near the trap)	−8

Then the robot, seeing different positions, can judge: "is this state good?" This is the Value Function.

10.5 Bellman equation: the core mathematical structure of reinforcement learning

This is one of the most important formulas in the whole mathematical system of reinforcement learning.

Bellman Expectation Equation

Starting from:

$$G_t = R_{t+1} + \gamma G_{t+1} \quad \text{Formula 5}$$

taking the conditional expectation on both sides, we get:

$$V^{\pi}(s) = E_{\pi}[R_{t+1} + \gamma V^{\pi}(S_{t+1}) | S_t = s]$$

The meaning of this formula is actually just splitting a long-term problem into:

how much I get now + how much the future is worth

that is:

$$V^{\pi}(s) = \frac{即时奖励}{现在} + \gamma \frac{未来价值}{未来}$$

This is the Bellman idea.

Why is it so important?

Because the original problem: $G_t = R_{t+1} + \gamma R_{t+1} + \gamma^2 R_{t+3} + \ldots\ldots$ is a very long future sequence. The Bellman equation compresses it into: {current one step} + {value of the next state}, i.e.:

$$G_t = R_{t+1} + \gamma V(s')$$

This is called: Bellman decomposition.

Physical meaning:

Imagine you are standing at a fork in the road. You do not need to simulate all 1000 future steps. You only need to ask:

"If I take this step, what do I gain immediately? After I go over there, how much is that position worth?"

Thus:

{current value} = {current reward} + {future value}

This is actually the fundamental reason reinforcement learning can be computed.

10.6 Q FUNCTION: WHY DO WE STILL NEED IT?

The Value Function only tells us $V(s)$, i.e., "is this state good?" But when we truly need to make a decision, we care more: "in this state, which action should I take is best?"

So we define the Q function (action-value function).

$$Q^{\pi}(s,a) = E_{\pi}[G_t | S_t = s, A_t = a] \quad \text{Formula 6}$$

Mathematical meaning

It computes: knowing we are currently in s and currently forced to take action a, then continuing to act according to the policy π, the expectation of the total future return.

So $V(s)$ answers: how is this state? While Q(s,a) answers: in this state, how is doing this action?

Physical meaning

For example, the robot is at: s={fork}

Then: Q(s,left) denotes:

If I go left now, how good will the future overall be? And: Q(s,right) denotes:

If I go right now, how good will the future overall be?

So the decision is very natural:

$$a^* = \underset{a}{argmax}\, Q(s,a)$$

that is:

choose the action with the largest Q value.

10.7 THE RELATIONSHIP BETWEEN V AND Q

This is another very important formula.

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s,a) \qquad \text{Formula 7}$$

Mathematical meaning: this is a weighted average. Suppose:

Q(s,left)=10

Q(s,right)=2

And the policy:

π(left|s)=0.7

π(right|s)=0.3

Then:

V(s)=0.7(10)+0.3(2)=7.6

The physical meaning can be understood as:

State value = the average result obtained by doing various actions according to the current behavioral habit. So Q(s,a) is like: "the score of this action," while V(s) is like: "the average score of this position under the current policy overall."

10.8 OPTIMAL VALUE FUNCTION

Now we are no longer satisfied with one particular policy. We want: among all possible policies, which is the best?

$$V^*(s) = \max_{\pi} V^{\pi}(s) \qquad \text{Formula 8}$$

Mathematical meaning: take the maximum over all possible policies. That is:

$$V^*(s) = \max_{\pi} E_{\pi}[G_t | S_t = s]$$

Physical meaning: it says that, if the robot is smart enough, starting from this state, how much long-term reward can it obtain at most? So $V^*(s)$ is a "theoretical highest score."

10.9 Optimal Q function

$$Q^*(s,a) = \max_{\pi} Q^{\pi}(s,a) \qquad \text{Formula 9}$$

Mathematical meaning

In the current state s, first execute action a, then in the future adopt the optimal policy.

Therefore: Q*(s,a) is the maximum expected return this action can achieve under optimal conditions.

Physical meaning

It is: "is doing this action now worth it?" So the optimal action:

$$\pi^*(s) = \arg\max_{a} Q^*(s,a)$$

This sentence is very important:

Reinforcement learning ultimately seeks a policy that maximizes long-term return.

10.10 Bellman Optimality Equation

Combine the previous ideas.

$$V^*(s) = \max_{a} E[R_{t+1} + \gamma V^*(S_{t+1}) | S_t = s, A_t = a] \qquad \text{Formula 10}$$

Mathematical meaning

It tells us: 最优价值 $= \max_{动作}$ [即时奖励] $+ \gamma$ [*未来最优价值*]

Note that max appears here, meaning: instead of averaging according to a fixed policy, choose the optimal action among the actions.

Physical meaning

For example, the robot comes to a fork:

Left: {immediate reward} + {future value}

Right: {immediate reward} + {future value}

The robot chooses the larger one. So:

{today's optimal choice depends on tomorrow's optimal value}

This is the core idea of dynamic programming.

10.11 WHY IS Q-LEARNING SO IMPORTANT?

Q-learning directly learns $Q^*(s,a)$ without needing to explicitly learn V(s).

Its core update formula is:

Q-learning: $\mathrm{Q}(s_t,a_t) \leftarrow \mathrm{Q}(s_t,a_t) + \alpha[r_{t+1} + \gamma \max_{\alpha'} Q(s_{t+1},\alpha') - Q(s_t,\alpha_t)]$ Formula 11

This is one of the most classic formulas in reinforcement learning.

First look at the thing inside the parentheses, defining:

$$\delta_t = r_{t+1} + \gamma \max_{\alpha'} Q(s_{t+1},\alpha') - Q(s_t,\alpha_t)$$

This is called the TD Error, the temporal-difference error.

Mathematical meaning: $\delta_t = 目标值 - 当前估计$, where the target: $r_{t+1} + \gamma \max_{\alpha'} Q(s_{t+1},\alpha')$, the current estimate: $Q(s_t,\alpha_t)$, so: $\delta = 新信息 - 旧预测$. Then $Q_{new} = Q_{old} + \alpha\delta$.

Physical meaning

Suppose the robot originally thought: "going left is good, worth 8," so: Q(s,left)=8.

After the robot walks over, it discovers: it turns out there was a goal with a reward of +n ahead. So it actually finds: $r_{t+1} + \gamma \max_{\alpha'} Q(s_{t+1},\alpha') = 12$, thus:$\delta = 12 - 8 = 4$

If $\alpha = 0.1$, then: Qnew=8+0.1(4)=8.4

The robot does not immediately think: Q=12, but only: 8->8.4, correcting gradually.

10.12 THE ESSENCE OF TD LEARNING

We can write the Q-learning update as: {new estimate} = {old estimate} + {learning rate}{prediction error}, i.e.:

$$\mathrm{Q} \leftarrow \mathrm{Q} + \alpha\delta$$

Mathematical meaning: this is actually very similar to many numerical optimization algorithms:$\theta_{new} = \theta_{old} + \text{correction}$

That is: reinforcement learning continuously uses the prediction error to correct its own function estimate.

Physical meaning: it is like a person learning to walk. He predicts: "stepping here should be fine." Then: "slap, he falls." So: {prediction} -> {actual feedback} -> {error} -> {correction}. Next time: "you cannot step here." This is the most intuitive physical picture of TD Learning.

10.13 EXPLORATION AND EXPLOITATION

Here comes a very practical problem. Suppose the robot now knows: Q(s,left)=8, Q(s,right)=5. Of course it will choose: {left}. But: if it has never tried the right side, how does it know whether the right side actually holds a treasure of +100? This is: Exploration vs Exploitation.

$\epsilon-$ greedy $\quad \pi(a|s)=\begin{cases} 随机动作,\epsilon \\ argmax_a Q(s,a), 1-\epsilon \end{cases}$ Formula 12

Mathematical meaning: with probability ϵ it performs random exploration, and with probability $1-\epsilon$ it chooses the current best action. For example: $\epsilon=0.1$ means: 90% of the time exploit current knowledge, 10% of the time explore unknown actions.

Physical meaning: it is like a person eating at a restaurant: if you always order your favorite dish: always order beef noodles, you may never know that the fish-fragrant shredded pork on the new menu is actually tastier. So occasionally you should: "try what you have not eaten." This is exploration.

10.14 FROM TABULAR Q TO NEURAL NETWORKS

The Q-learning above has an implicit assumption: the number of states is not very large. We can directly build a Q(s,a) table.

For example:

State	Left	Right
s1	5	2
s2	8	4
s3	1	9

But if the state is an image, e.g., 84*84*3, then the state space is huge and it is completely impossible to build a Q table.

So we approximate with a neural network: $Q(s,a,\theta)$, where θ is the neural network parameter.

10.15 REINFORCEMENT LEARNING SUDDENLY BECOMES A FUNCTION-APPROXIMATION PROBLEM

$$Q(s,a) \approx Q(s,a,\theta)$$ Formula 13

Mathematical meaning: we no longer store the Q value of every state. Instead, we look for a function:

$$f_\theta(s,a)$$

such that:

$$f_\theta(s,a) \approx Q^*(s,a)$$

So reinforcement learning becomes: finding a parameter θ such that the neural network can approximate the optimal value function. θ i.e., the neural network weights.

Physical meaning: in the past, the robot needed to memorize: "this position scores 8." Now the robot learns a rule: "the closer to the goal and the farther from the trap, the higher the value usually is." Therefore it goes from {memory} to {generalization}.

10.16 The loss function of Deep Q-Network

Q-learning can construct a target: $y = r + \gamma \max_{a'} Q(s', a', \theta')$

and then hope: $Q(s, a, \theta) \approx y$

So define the loss:

$$L(\theta) = \frac{1}{2}[y - Q(s, a, \theta)]^2 \qquad \text{Formula 14}$$

Mathematical meaning: this is an ordinary squared error $\text{Loss} = \frac{1}{2}(\text{目标} - \text{预测})^2$**, and then use gradient descent:**

$$\theta \leftarrow \theta - \eta \nabla_\theta L(\theta)$$

i.e., the neural network continuously adjusts its parameters so that the prediction gets closer and closer to the empirical target.

10.17 Policy Gradient: why can we directly learn the policy?

Earlier we learned Q(s,a) and then chose actions from Q. Another route is to directly learn $\pi_\theta(a|s)$, i.e., directly let the neural network decide which action to take.

$$J(\theta) = E_{\tau \sim \pi_\theta}[G(\tau)] \qquad \text{Formula 15}$$

where τ denotes a whole trajectory:

$$\tau = (s_0, a_0, r_1, s_1, \dots)$$

Mathematical meaning: we want $J(\theta)$ **to be maximized. That is: let the average cumulative reward of the trajectories produced by the policy be maximized. That is:** $\max_\theta J(\theta)$**.**

Physical meaning: this time we no longer ask "which action has the highest value?", but directly ask: "how should I adjust my behavioral habit so that I can get more rewards in the future?" For example, the robot originally: P(left)=0.5, P(right)=0.5. After training: P(left)=0.9, P(right)=0.1; it directly changes its own action probabilities.

10.18 The most core formula of Policy Gradient

$$\nabla_\theta J(\theta) = E[\nabla_\theta \log \pi_\theta(a|s) G_t] \qquad \text{Formula 16}$$

This is the famous Policy Gradient / REINFORCE form.

Mathematical meaning: if a certain action ultimately brings a high $\nabla_\theta log\pi_\theta(a|s)$ G(t), then the neural network will be pushed to prefer choosing this action in the future. Conversely, if G(t) is very low, this action will be suppressed.

Physical meaning: imagine training a child. The child makes a decision: "I want to go left." Result: +100. You tell him: "good, do this more in the future." So: P(left) increases. If: -100, you tell him: "this road is not good, do this less in the future." So: P(left) decreases.

Policy Gradient is essentially: {reward good behaviors and weaken bad behaviors}

10.19 ACTOR-CRITIC: COMBINING THE VALUE FUNCTION AND THE POLICY FUNCTION

Now we have two sets of ideas:

Critic: learns V(s), or Q(s,a); it is responsible for evaluating behaviors.

Actor: learns $\pi(a|s)$; it is responsible for choosing behaviors.

Thus: $\boxed{\text{Actor}\} + \text{Critic}}$

Advantage function $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ Formula 17

Mathematical meaning: it compares the value of taking an action $Q^\pi(s,a)$ with the general value of this state $V^\pi(s)$. If: $A^\pi(s,a) > 0$, it means this action is better than the average level. If $A^\pi(s,a) < 0$, it means this action is worse than the average level.

The advantage function is asking: "how much better is this action than the average level of this state?"

10.20 THE INTUITION OF ACTOR-CRITIC

Actor: $\pi_\theta(a|s)$, responsible for "what should I do?"

Critic: V(s) is responsible for evaluating "how well did you do?"

Thus: {Actor decision} ->{environment feedback} ->{Critic evaluation} ->{Actor improvement}

This is the basic structure of many modern reinforcement learning algorithms.

10.21 PUTTING THE WHOLE MATHEMATICAL SYSTEM OF REINFORCEMENT LEARNING TOGETHER

Now we can connect the previous formulas into a chain.

At the very beginning:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Define the state value: $V^\pi(s) = E[G_t|S_t = s]$

Define the action value: $Q^\pi(s,a) = E[G_t|S_t = s, A_t = a]$

Obtain the Bellman equation:$V^{\pi}(s) = E[R + \gamma V^{\pi}(s')]$

In the optimal case:$V^{*}(s) = max_{a}E[R + \gamma V^{*}(s')]$

Further:$Q^{*}(s,a) = E[R + \gamma max_{a'}Q^{*}(s',a')]$

Then we can perform Q-learning:$Q \leftarrow Q + \alpha[r + \gamma max_{a'}Q' - Q]$

If the state space is huge:$Q(s,a) \rightarrow Q(s,a;\theta)$

Then optimize with a neural network:$\theta \leftarrow \theta - \eta\nabla_{\theta}L$

The other route directly optimizes:$max_{\theta}J(\theta)$

Obtaining:$\nabla_{\theta}J = E[\nabla_{\theta}\log\pi_{\theta}(a|s) \cdot G_{t}]$

Finally forming: Actor-Critic

10.22 UNDERSTANDING REINFORCEMENT LEARNING IN ONE SENTENCE

Reinforcement learning can be condensed as: try actions → obtain rewards → evaluate results → revise the policy → try again.

Mathematically it is: Policy → Trajectory → Return → Value → Error → Update.

That is: $\pi \rightarrow G \rightarrow V/Q \rightarrow \delta \rightarrow \theta \rightarrow \pi_{new}$, which is in fact the mathematical closed loop of reinforcement learning.

The most important "physical picture": if you remember only one diagram, remember: state s selects action a via $\pi(a|s)$, the environment returns reward r and new state s'; then (s,a,r,s') is used to compute the error and update the value and the policy.

The ultimate goal: $max_{\pi}E_{\pi}[\sum_{t=0}^{\infty}\gamma^{t}R_{t+1}]$, which is the most fundamental mathematical objective of reinforcement learning.